Aztec Labs Polynomial Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Farouk

qpzm

May 8, 2026

Contents

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The `polynomials` module is the foundational data-structure and arithmetic layer of Barretenberg's UltraHonk proving system. It provides the polynomial types, field-arithmetic primitives, evaluation domains, and lookup-table precomputations that Sumcheck, Gemini, Shplonk, KZG, IPA, and the ZK-Sumcheck Libra protocol all build on.

Production usage: every UltraHonk proof touches this module. Wire/selector/permutation polynomials are instances of `Polynomial<Fr>`; Sumcheck's round univariates are `UnivariateCoefficientBasis` + `Univariate`; the multilinear equality polynomial in Sumcheck is `ProverEqPolynomial` / `VerifierEqPolynomial`; Gemini's fold polynomials, Shplonk's batched quotient, and IPA's final reduction all run through `Polynomial::operator+=`, `add_scaled`, `factor_roots`, `shifted`, and `full`. ECCVM and Translator (Grumpkin curve) use the same types plus the `SmallSubgroupIPA` + `BarycentricData`-backed Libra protocol for ZK. The module is explicitly instantiated only for `bb::fr` (BN254 scalar) and `grumpkin::fr`.

4.1 Module File Overview

File	Role
`backing_memory.cpp/hpp`	Heap + optional file-backed mmap allocator for polynomial storage
`shared_shifted_virtual_zeroes_array.hpp`	Sparse array primitive: `[start_, end_)` backed, `[0, virtual_size_)` logically valid with zeros outside `[start_, end_)`
`polynomial.cpp/hpp`	Main `Polynomial<Fr>` type; coefficient arithmetic, MLE evaluation, shifts, reversals, interpolation
`polynomial_arithmetic.cpp/hpp`	Free-function primitives: Horner evaluation, FFT/iFFT (BN254 only), Lagrange interpolation, `factor_roots`, linear product

File	Role
evaluation_domain.cpp/hpp	FFT domain with precomputed roots-of-unity tables (BN254) / root=1 stub (Grumpkin)
barycentric.hpp	Compile-time and runtime Barycentric interpolation data (domain, denominator_inverses, full_numerator_values)
univariate_coefficient_basis.hpp	Degree ≤ 2 polynomials in monomial basis with Karatsuba-friendly encoding for Sumcheck relations
eq_polynomial.hpp	Multilinear equality polynomial eq(X, r) (prover full-table form and verifier single-point form)

Closely related types defined outside this module: gate_separator.hpp (pow-β polynomial; delegated to by ProverEqPolynomial::construct), univariate.hpp (Lagrange-basis counterpart to UnivariateCoefficient-Basis), row_disabling_polynomial.hpp (ZK-Sumcheck masking).

4.2 Storage Model — Three-Layer Stack

1. BackingMemory<Fr>: owns a raw Fr* via either heap (new Fr[] + custom-deleter shared_ptr<Fr[]>) or a file-backed mmap. File-backed mode is gated by BB_SLOW_LOW_MEMORY=1 and uses O_EXCL | 0600 | MAP_-PRIVATE for security. Allocation does **not** zero memory — callers zero in parallel.
2. SharedShiftedVirtualZeroesArray<T>: a thin struct holding start_, end_, virtual_size_ (size_t) plus BackingMemory<T>. Only indices in [start_, end_) consume memory; reads at [0, start_) or [end_, virtual_size_) return a static zero reference. This sparse layout lets Honk encode "wire polynomial shifted by NUM_ZERO_ROWS=1" (start_=1) and "polynomial with lots of trailing zeros" (end_ < virtual_size_) without allocating the unused regions.
3. Polynomial<Fr>: wraps one SharedShiftedVirtualZeroesArray<Fr> member (coefficients_). Provides the public API: constructors, MLE evaluation, Horner evaluation, in-place +=/-=/*=, add_scaled, shifted(), reverse(), full(), factor_roots(), interpolation. Also provides the share() shallow-copy primitive (ref-count bump).

Invariant maintained by the public API: start_ <= end_ <= virtual_size_. Direct aggregate-initialization of a SharedShiftedVirtualZeroesArray can break it, but no production caller does so.

4.3 Core Polynomial Operations — Where Each is Used

Operation	Function(s)	Protocol use
MLE evaluation	Polynomial::evaluate_mle(u, shift), _evaluate_mle	Sumcheck opening step (prover and verifier via Gemini); shift=true handles Honk's shifted-witness evaluation
Univariate Horner eval	Polynomial::evaluate(z), polynomial_arithmetic::evaluate	Gemini fold-polynomial claimed evaluations; KZG opening-pair evaluations
Synthetic division by (X - r)	Polynomial::factor_roots(r), polynomial_arithmetic::factor_-roots	KZG opening quotient; Shplonk per-claim quotient; Goblin merge prover

Operation	Function(s)	Protocol use
Left-shift (divide by X)	`Polynomial::shifted()`	Honk shifted-witness polynomials (wires, z_perm); Gemini fold polynomials
Multi-polynomial batching	`operator+=`, `operator-=`, `add_scaled`	Gemini batch-polynomial construction; Shplonk batched quotient Q; Sumcheck round polynomial accumulation
Lagrange interpolation	`Polynomial(interpolation_points, evaluations, vsize)`, `compute_efficient_interpolation`, `compute_linear_polynomial_product`	SmallSubgroupIPA challenge polynomial; ZK-Sumcheck Libra concatenated polynomial (Grumpkin path)
FFT / iFFT	`fft_inner_parallel`, `ifft`	ZK-Sumcheck Libra monomial-form conversion (BN254 path only, over SUBGROUP_SIZE = 256); coset-FFT variant is dead code
Zero-initialization	`Polynomial(size, vsize)` (parallel memset); `Polynomial(..., DontZeroMemory::FLAG)`	Every fresh polynomial allocation; performance-sensitive callers opt into `DontZeroMemory`

4.4 Types Used by Sumcheck

- `UnivariateCoefficientBasis<Fr, LENGTH, has_a0_plus_a1>` holds degree ≤ 2 polynomials $a_0 + a_1 X + a_2 X^2$ in monomial basis, with the `has_a0_plus_a1` flag enabling a Karatsuba-trick multiplication based on $(a_0 + a_1)(b_0 + b_1) - a_0 b_0 = a_0 b_1 + a_1 b_0 + a_1 b_1$. When `has_a0_plus_a1 = true`, the sum $a_0 + a_1$ is cached in `coefficients[2]`, so the subsequent multiply skips recomputing it — one saved field addition per precomputed operand. The degree-2 product (from multiplying two degree-1 operands) stores `coefficients[1]` $= a_1 + a_2$ rather than a_1 alone — a forward-difference encoding, chosen so that `coefficients[0] + coefficients[1]` directly equals $P(1) = a_0 + a_1 + a_2$. Conversion back to Lagrange form over $\{0, 1, 2\}$ then costs one addition at $X = 1$ instead of two, with $P(0) =$ `coefficients[0]` and `coefficients[2]` $= a_2$ still giving the monomial leading coefficient.
- `Univariate<Fr, LENGTH>` is the Lagrange-basis counterpart over the consecutive-integer domain $\{0, 1, \ldots,$ domain_end $- 1\}$. `extend_to<EXTENDED_DOMAIN_END>()` uses hand-optimized direct formulas for LENGTH $\in \{2, 3, 4\}$ and falls back to `BarycentricData` for LENGTH ≥ 5; `evaluate(u)` always uses `BarycentricData`.
- `BarycentricData<Fr, domain_end, num_evals>` precomputes the domain $\{0, \ldots,$ big_domain $- 1\}$, the Lagrange denominators $d_j = \prod_{k \neq j}(x_j - x_k)$, the inverse products $1/\big(d_j(x_k - x_j)\big)$ for $k \in$ [domain_end, num_evals), and the full numerator $M(x_k) = \prod_j (x_k - x_j)$. For native `bb::field` types this is all `static constexpr` (compile-time); for stdlib `field_t` it is `inline static const` (startup-time). Batch inversion via Montgomery's trick skips zeros (entries at $k <$ domain_end, which are never read).
- `ProverEqPolynomial<FF>::construct(r, log_num_monomials)` builds the full 2^d-entry coefficient table of $\mathrm{eq}(X, r)$. The optimal path transforms $\mathrm{eq}(X, r) = C \cdot \prod_i (1 + \beta_i X_i)$ with $C = \prod_i (1 - r_i)$, $\gamma_i = r_i/(1 - r_i)$, and $\beta_i := \gamma_i - 1 = (2r_i - 1)/(1 - r_i)$, then delegates to `GateSeparatorPolynomial::compute_beta_products`. Falls back to direct construction when any $r_i = 1$ (i.e., $C = 0$). The base definition is

$$\mathrm{eq}(X, r) = \prod_{i=0}^{d-1} \big((1 - r_i)(1 - X_i) + r_i X_i\big).$$

- `VerifierEqPolynomial<FF>::evaluate(u)` computes $\mathrm{eq}(u, r) = \prod_i (b_i + u_i a_i)$ with $a_i = 2r_i - 1$, $b_i = 1 - r_i$

— a streaming $O(d)$ evaluation that the verifier uses at the Sumcheck random point without constructing the full table.

4.5 Evaluation Domain and FFT

`EvaluationDomain<Fr>` packages everything needed to run an in-place radix-2 Cooley-Tukey FFT over a multiplicative subgroup of size `n = 2^k`: the primitive `n`-th root ω, ω^{-1}, a coset generator `g`, g^{-1}, a shared `roots` buffer of `2n` field elements (forward roots in the first half, inverse in the second), and per-round pointer tables `round_roots` / `inverse_round_roots`. Lookup tables are built lazily by `compute_lookup_table()` after construction.

Production usage is narrow:

- Only BN254 `fr` is used for real FFTs. Grumpkin `fr` has 2-adicity 1, so the primary constructor sets `root = Fr::one()` — an FFT with $\omega = 1$ is a no-op, which is fine because no code path actually calls FFT on a Grumpkin-typed domain.
- The only production-reachable `EvaluationDomain<bb::fr>(SUBGROUP_SIZE, SUBGROUP_SIZE)` call sites are in `ZKSumcheckData::create_interpolation_domain` and `SmallSubgroupIPAProver::compute_monomial_coefficients`, both at `SUBGROUP_SIZE = 256` (2^8). The 28-bit max circuit size is irrelevant here — the subgroup used by the Libra ZK mask is fixed at 2^8, far below FFT precision limits.
- `EvaluationDomain<grumpkin::fr>` is instantiated as a template (so the code compiles) but never constructed with non-zero `domain_size` in production; only the default-constructed empty state is held as a member of ECCVM/Translator variants of `ZKSumcheckData`.
- Grumpkin's FFT gap is why ECCVM/Translator's ZK-Sumcheck Libra path uses direct Lagrange interpolation (`compute_efficient_interpolation` over an 87-element multiplicative subgroup) instead of monomial-Lagrange conversion via iFFT. The BN254 path, which has 2-adicity 28, uses iFFT over the 256-element subgroup instead.

4.6 Key Design Properties

- No virtual zeros in the FFT: FFTs always operate on a dense buffer of `domain.size` elements. The virtual-zeros model lives one level up at `Polynomial` / `SharedShiftedVirtualZeroesArray`.
- Shallow sharing is explicit: `share()` is the one path that produces a refcount-shared alias. `shifted()` also shares memory but re-parents `start_`/`end_` by -1 to present a "divided by X" view.
- Deep copies go through `_clone`: `polynomial.cpp`'s `_clone(array, right_ext, left_ext)` allocates a fresh `BackingMemory`, zero-pads the extensions, and `memcpy`s the existing data. The copy constructor delegates to the size-extending copy constructor `Polynomial(other, target_size)`, which calls `_clone`; copy assignment and `full()` call `_clone` directly. `_clone` is the hub for *zero-extending* deep copies specifically — other copy-ish operations (`share`, `shifted`, span / interpolation constructors, `reverse`, moves) have their own, non-`_clone` implementations.
- Memory is never zeroed by the allocator: `BackingMemory::allocate(size)` returns uninitialized memory (raw `new Fr[]` without value-init parens). Every callsite either `memset`s in `Polynomial`'s constructor (optionally parallel), `memcpy`s into the buffer, or writes every element via arithmetic. Callers that want to skip the zero-init pay the `DontZeroMemory::FLAG` tax.
- Move semantics are lightly enforced: `BackingMemory`'s custom move correctly nulls the source's `raw_data`, but `Polynomial`'s and `SharedShiftedVirtualZeroesArray`'s defaulted moves leave scalar fields (`start_`, `end_`, `virtual_size_`) unchanged on the source. After `B = std::move(A)`, `A` looks valid to scalar queries (`A.size() > 0`) but `A.data() == nullptr`, so an access through `A.data()[i]` dereferences null. This is the root cause of the `EvaluationDomain` and `Polynomial` move-semantics findings.

4.7 Mathematical Correctness

Each of the core algorithms was verified against its mathematical definition:

- `_evaluate_mle` — standard hypercube fold tmp$[i] = c_{2i} + u \cdot (c_{2i+1} - c_{2i})$; the `ALLOW_ONE_PAST_READ=1` shim allows the shifted fold to read one index past `virtual_size` (returning zero) when dim $= n$.
- `factor_roots` — in-place synthetic division: the recurrence $b_i = (a_i - b_{i-1}) \cdot (-r)^{-1}$ fills slots $[0, n-2]$ with the quotient's coefficients, and slot $n-1$ is zeroed because $q(X) = p(X)/(X-r)$ has degree one less than p. Correct only when $p(r) = 0$; see the `factor_roots` remainder-masking finding for the concern when this precondition is violated.
- `compute_efficient_interpolation` — barycentric Lagrange interpolation with Montgomery batch inversion; separate branch for "0 in domain" uses $N(X)/X$ (since $N(X)$ has no constant term then). The formula:

$$F(X) = N(X) \cdot \sum_i \frac{y_i}{d_i(X - x_i)}.$$

- `fft_inner_parallel` — Cooley-Tukey radix-2 decimation-in-time with bit-reversed input, parallelized across `num_threads` chunks. - `compute_beta_products` / `ProverEqPolynomial::construct` — fills the $\mathrm{eq}(X, r)$ coefficient table in $O(2^d)$ by deriving each entry from its "previous-bit-pattern" sibling in $O(1)$. - `UnivariateCoefficientBasis::operator*` Karatsuba — result coefficients$[1] = (a_0 + a_1)(b_0 + b_1) - a_0 b_0 = a_0 b_1 + a_1 b_0 + a_1 b_1$, matching the Univariate-side conversion evaluations$[1] = a_0 + (a_1 + a_2)$ (forward-difference encoding).

4.8 Soundness Argument (Prover-Verifier Contract)

The polynomial module is not itself a constraint system — it supplies polynomial primitives to higher-level protocols. Its contribution to soundness is:

1. Correct MLE evaluation: Sumcheck's soundness reduces the multivariate claim to a univariate claim at a random point. `_evaluate_mle` must agree with the verifier's eval at the same point. Verified by the fold identity and correct handling of the `shift`, `dim < n`, and virtual-zeros cases.
2. Correct FFT/iFFT: ZK-Sumcheck Libra requires a monomial-Lagrange conversion that is exactly inverse. Verified by the `ifft_consistency` test (recovers coefficients from evaluations).
3. Correct Lagrange interpolation: SmallSubgroupIPA and ZK-Sumcheck Libra construct challenge polynomials and Libra polynomials via `compute_efficient_interpolation`, which must produce the unique polynomial through the given points. The distinct-points precondition is not asserted, but all production callers satisfy it by construction because they use distinct subgroup elements.
4. Correct equality polynomial: `eq(X, r)` is used as a Boolean selector. Both prover (`ProverEqPolynomial::construct`, building the full `2^d` table) and verifier (`VerifierEqPolynomial::evaluate`, streaming O(d) at a single point) must agree at Boolean inputs (Kronecker delta) and at the random Sumcheck point.
5. Correct Barycentric extension: Sumcheck round univariates of degree 1-6 are extended to larger domains (e.g., `EXTENDED_DOMAIN_END` for the check) using `BarycentricData`. Compile-time `constexpr` construction ensures the precomputed weights cannot be tampered with at runtime.

All five are mathematically verified. The audit found no soundness-breaking bug. All open findings are either latent safety (`Low` — require precondition violation or specialized deployment mode) or hygiene (`Informational`). Both the `evaluate_mle` SIGSEGV on a single-coefficient polynomial and the `get_scratch_space` shared buffer under concurrent FFTs carry a `Low` rating because neither is reachable through a current production caller.

4.9 Module-Level Key Constants

Constant	Value	Where
`NUM_ZERO_ROWS`	1	`constants.hpp` — leading zeros required for shiftable Honk witnesses

Constant	Value	Where
`CONST_SIZE_PROOF_LOG_N`	28	UltraHonk max `log_circuit_size` (defines the FFT size upper bound)
`SUBGROUP_SIZE` (BN254)	256 (= 2^8)	ZK-Sumcheck Libra / SmallSubgroupIPA subgroup
`SUBGROUP_SIZE` (Grumpkin)	87	ECCVM / Translator subgroup — not a power of 2 (uses direct interpolation, not FFT)
`ALLOW_ONE_PAST_READ`	1	`_evaluate_mle` shim for shifted MLE's out-of-virtual-size read

5 Audit Scope

The audit scope was limited to:

```
aztec-packages/barretenberg/cpp/src/barretenberg
 polynomials/backing_memory.cpp
 polynomials/backing_memory.hpp
 polynomials/barycentric.hpp
 polynomials/eq_polynomial.hpp
 polynomials/evaluation_domain.cpp
 polynomials/evaluation_domain.hpp
 polynomials/polynomial.cpp
 polynomials/polynomial.hpp
 polynomials/polynomial_arithmetic.cpp
 polynomials/polynomial_arithmetic.hpp
 polynomials/shared_shifted_virtual_zeroes_array.hpp
 polynomials/univariate_coefficient_basis.hpp
```

6 Executive Summary

Over the course of 8 days, the Cyfrin team conducted an audit on the Polynomial code provided by Aztec Labs. The findings consist of 15 Low severity issues with the remainder being informational.

Cyfrin conducted a targeted review of the Barretenberg `polynomials` module — twelve C++ source files that together provide the polynomial data structures, FFT / interpolation arithmetic, and specialised sumcheck primitives consumed by every prover and verifier in the Aztec proving stack. The module is a pure cryptographic primitives library with no on-chain component, no admin roles, and no governance surface; the value being protected is the soundness of proofs that ultimately settle on Ethereum L1.

The review focused on the areas where a silent miscomputation would translate into a soundness break or release-mode memory corruption: index arithmetic in `SharedShiftedVirtualZeroesArray` and `Polynomial`; the FFT bit-reversal, butterfly, and root-indexing code; the two-branch Lagrange interpolation with its zero-in-domain special case; `factor_roots` and its callers; the optimal/fallback split inside `ProverEqPolynomial`; the Karatsuba-based `UnivariateCoefficientBasis` and its `a+a` invariant; and the storage stack's CAS-based budget tracking, mmap lifecycle, and `parse_size_string` env-var handling.

The audit revealed a well-implemented and carefully engineered module with a mature set of core algorithms. No critical, high, or medium severity issues were identified. Opportunities for improvement cluster into a handful of themes. The largest group concerns **unchecked preconditions on helper APIs** — `factor_roots` silently masks the remainder when the supplied value is not a root, `compute_efficient_interpolation` produces wrong output on duplicate x-coordinates, `ProverEqPolynomial::construct` does not enforce the challenge-count precondition,

and the interpolation constructor does not assert that evaluations and x-coordinates are the same length — each of which could produce a silently incorrect polynomial if a caller drifted out of contract. A second group concerns **debug-only bounds and invariant checks** (`set`, `at`, `operator[]`, and the `SharedShiftedVirtualZeroesArray` range invariants) that compile away in release builds, creating potential for silent heap corruption if a future caller violated them. A third group concerns **move and copy semantics** in `Polynomial` and `EvaluationDomain`, where moved-from and self-assignment paths can leave structurally inconsistent state or permanently destroy lookup tables. A final group concerns **latent concurrency and edge-case bugs** — a `thread_local` gap in the FFT scratch buffer, an aliasing hazard when `fft_inner_parallel` is invoked with `coeffs == target`, a SIGSEGV on a degenerate zero-variable MLE, an underflow in `Polynomial::random` with an out-of-range `start_index`, and an overflow in `parse_size_string` — each currently unreachable but cheap to harden. The remaining findings are informational: efficiency and ergonomics improvements around `Polynomial::full()` double-cloning, `compute_linear_polynomial_product`'s asymptotic, and preserved Karatsuba precomputation across `UnivariateCoefficientBasis` operators.

Overall the module presents a strong security posture. The recommendations in this report are best characterised as defensive hardening and contract-strengthening: promoting key debug assertions into release-mode checks, asserting caller preconditions on helper APIs that today fail silently, tightening move and assignment semantics, and closing a small number of latent edge cases before new call sites can reach them.

Summary

Project Name	Polynomial
Repository	aztec-packages
Commit	e88ecdf581db...
Audit Timeline	Apr 13th - Apr 22nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	15
Informational	8
Gas Optimizations	0
Total Issues	23

Summary of Findings

[L-01] `factor_roots` masks remainder — silent wrong result on violated precondition	Resolved
[L-02] `compute_efficient_interpolation` silently produces wrong output on duplicate evaluation points	Resolved
[L-03] `evaluate_mle` crashes (SIGSEGV) on single-coefficient polynomial	Resolved

[L-04] Interpolation constructor does not assert equal sizes of interpolation points and evaluations	Resolved
[L-05] `get_scratch_space` shared buffer unsafe under concurrent FFTs	Resolved
[L-06] `parse_size_string` does not check for overflow in `value * multiplier`	Resolved
[L-07] `EvaluationDomain` copy constructor off-by-one for size=2 domains	Resolved
[L-08] `UnivariateCoefficientBasis` `operator==` compares unused `coefficients[2]` for `domain_end=2`	Resolved
[L-09] Signed integer overflow in `_evaluate_mle` bit-shift expressions	Resolved
[L-10] `EvaluationDomain` move-assignment self-assignment permanently destroys lookup tables	Resolved
[L-11] `Polynomial::random(size, start_index)` underflows when `start_index > size`	Resolved
[L-12] `ProverEqPolynomial::construct` returns `[0]` instead of `[1]` for empty challenges	Resolved
[L-13] `EvaluationDomain` move constructor leaves inconsistent state	Resolved
[L-14] `Polynomial` defaulted move leaves inconsistent state	Resolved
[L-15] `ProverEqPolynomial::construct` fast path does not enforce `challenges.size() == log_num_monomials`	Resolved
[I-1] `SharedShiftedVirtualZeroesArray` invariant enforcement gap	Resolved
[I-2] `set()`, `at()`, and `operator[]` (mutable) use debug-only bounds checks allowing silent heap corruption in release	Acknowledged
[I-3] `backing_memory.cpp` preprocessor guard inconsistent with header	Acknowledged
[I-4] `fft_inner_parallel` unsafe when `coeffs == target`	Resolved
[I-5] `compute_linear_polynomial_product` is O(n^3) instead of O(n^2)	Resolved
[I-6] `UnivariateCoefficientBasis` loses Karatsuba precomputation after arithmetic	Acknowledged
[I-7] `Polynomial::full()` performs unnecessary double-clone	Resolved
[I-8] scalar `operator*=` unnecessarily restricted by `requires(!has_a0_plus_a1)`	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 `factor_roots` masks remainder — silent wrong result on violated precondition

Description: In polynomial_arithmetic.hpp:56, `factor_roots` performs synthetic division assuming $(X - r) \mid p(X)$. After the division loop, it unconditionally zeroes the final coefficient:

```
polynomial[size - 1] = Fr::zero();  // unconditionally zeroes the leading coefficient slot
```

https://github.com/AztecProtocol/aztec-packages/blob/main/barretenberg/cpp/src/barretenberg/polynomials/polynomial_arithmetic.hpp#L107

The division loop (lines 99-105) computes quotient coefficients $b_0, \ldots, b_{n-2}$ in-place, storing the last value in `temp`. It never touches `polynomial[size-1]`, which still holds the original a_{n-1}. For exact division, $a_{n-1} = b_{n-2}$ (i.e., `polynomial[size-1] == temp`). When the precondition is violated, these differ — but line 107 unconditionally zeroes the slot without checking, silently producing a wrong quotient with no indication of error.

Impact: Low. Called 7 times in proof-critical path (all prover-side). All callers correctly satisfy the precondition — KZG subtracts the evaluation before quotienting, Shplonk/Gemini evaluations are computed from the polynomial itself, and Merge prover evaluations are derived directly. A failure requires a future trusted-caller bug (not external input), and the result would be verifier rejection, not a soundness break. The assert costs 1 field comparison and would make such a bug fail loudly instead of silently.

All 7 call sites (prover only):

1. kzg/kzg.hpp:56 — KZG Prover
   - Root: `pair.challenge`
   - Precondition: `quotient.at(0) -= pair.evaluation`
2. shplonk/shplonk.hpp:77 — Shplonk Prover
   - Root: `-claim.opening_pair.challenge`
   - Precondition: `tmp.at(0) -= gemini_fold_pos_evaluations[fold_idx]`
3. shplonk/shplonk.hpp:86 — Shplonk Prover
   - Root: `claim.opening_pair.challenge`
   - Precondition: `tmp.at(0) -= claim.opening_pair.evaluation`
4. shplonk/shplonk.hpp:102 — Shplonk Prover (Libra)
   - Root: `claim.opening_pair.challenge`
   - Precondition: `tmp.at(0) -= claim.opening_pair.evaluation`
5. shplonk/shplonk.hpp:114 — Shplonk Prover (Sumcheck)
   - Root: `claim.opening_pair.challenge`
   - Precondition: `tmp.at(0) -= claim.opening_pair.evaluation`
6. goblin/merge_prover.cpp:84 — Merge Prover
   - Root: `kappa`
   - Precondition: `shplonk_batched_quotient.at(0) -= challenge * eval` (in loop)
7. goblin/merge_prover.cpp:91 — Merge Prover
   - Root: `kappa_inv`
   - Precondition: `reversed_batched_left_tables_copy.at(0) -= evals.back()`

Proof of Concept: AuditPoC_P17_FactorRootsMasksRemainder — saves the original a_{n-1} before calling `factor_roots`, then compares it with b_{n-2} (`poly[n-2]` after the call). For exact division these match; for violated preconditions they differ, which is exactly what our proposed assert catches.

```
TYPED_TEST(PolynomialTests, AuditPoC_P17_FactorRootsMasksRemainder)
{
    using FF = TypeParam;

    constexpr size_t n = 3;

    // Case 1: Exact division - p(X) = X^2 - 1 = (X-1)(X+1), root = 1
    {
        std::array<FF, n> poly = { -FF(1), FF(0), FF(1) }; // [-1, 0, 1]
        FF original_leading = poly[n - 1];                  // a_{n-1} = 1

        polynomial_arithmetic::factor_roots(std::span<FF>(poly), FF(1));

        // After division: poly[n-2] = b_{n-2} (last quotient coeff)
        // For exact division: a_{n-1} == b_{n-2}
        FF b_last = poly[n - 2];
        EXPECT_EQ(original_leading, b_last);
    }

    // Case 2: Non-exact division - p(X) = X^2 + 1, root = 1, p(1) = 2  0
    {
        std::array<FF, n> poly = { FF(1), FF(0), FF(1) }; // [1, 0, 1]
        FF original_leading = poly[n - 1];                 // a_{n-1} = 1

        FF eval_at_root = polynomial_arithmetic::evaluate(poly.data(), FF(1), n);
        EXPECT_NE(eval_at_root, FF(0));

        polynomial_arithmetic::factor_roots(std::span<FF>(poly), FF(1));

        // After division: a_{n-1}  b_{n-2} - proposed assert catches this
        FF b_last = poly[n - 2];
        EXPECT_NE(original_leading, b_last);
    }
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P17*"
```

```
P-17 Case 1 (exact): a_{n-1}=..01, b_{n-2}=..01       → proposed assert PASSES
P-17 Case 2 (non-exact): a_{n-1}=..01, b_{n-2}=..p-1   → proposed assert CATCHES non-divisibility
```

Recommended Mitigation: Replace polynomial_arithmetic.hpp:107:

```
- polynomial[size - 1] = Fr::zero();
+ BB_ASSERT(polynomial[size - 1] == temp);  // verify a_{n-1} == b_{n-2} (exact divisibility)
+ polynomial[size - 1] = Fr::zero();
```

Aztec: Fixed in 80131f6.

Cyfrin: Verified.

7.1.2 `compute_efficient_interpolation` silently produces wrong output on duplicate evaluation points

Description: In polynomial_arithmetic.cpp:249, `compute_efficient_interpolation` performs Lagrange interpolation from evaluation points and values. When two evaluation points are equal, the Lagrange denominator

$$d_i = \prod_{j \neq i} (x_i - x_j)$$

becomes 0 for the duplicated indices. batch_invert skips zero entries, leaving $d_i^{-1} = 0$. The contributions from the duplicated points are silently zeroed out, producing an incorrect interpolation with no error or assertion.

```
// Line 296-301: compute denominators
for (size_t j = 0; j < n; ++j) {
    if (j == i) continue;
    roots_and_denominators[n + i] *= (evaluation_points[i] - evaluation_points[j]);
    // If evaluation_points[i] == evaluation_points[j], this multiplies by 0
    // → d_i = 0 → batch_invert skips → contribution silently zeroed
}

// Line 305: batch invert all denominators (zeros are skipped)
Fr::batch_invert(roots_and_denominators.data(), 2 * n);
```

batch_invert skips zeros by design — when an element is zero, it is excluded from the running product and its result slot is left as 0:

```
// field_impl.hpp:430
if (coeffs[i].is_zero()) {
    skipped[i] = true;     // zero excluded from accumulator
}
```

Other callers (e.g., barycentric evaluation) legitimately rely on this skip-zero behavior. The fix belongs in `compute_efficient_interpolation` — add a distinct-points assertion before calling `batch_invert`.

Impact: Low. Duplicate points make Lagrange interpolation mathematically undefined — this is a precondition violation, not a protocol flaw. All production callers use fixed subgroup domains (`{1, g, ..., g^(n-1)}`), which are distinct by construction. There is no current path for an external actor to supply duplicate points. The assert is a defensive check against future misuse.

Proof of Concept: Added AuditPoC_P07_InterpolationDuplicatePoints to `polynomial_arithmetic.test.cpp`:

```
TYPED_TEST(PolynomialTests, AuditPoC_P07_InterpolationDuplicatePoints)
{
    using FF = TypeParam;

    // Create a known polynomial: f(x) = 3x^2 + 2x + 1
    constexpr size_t n = 3;
    std::array<FF, n> poly = { FF(1), FF(2), FF(3) };

    // Case 1: Distinct points - should interpolate correctly
    {
        std::array<FF, n> x = { FF(1), FF(2), FF(3) };
        std::array<FF, n> src;
        for (size_t i = 0; i < n; i++) {
            src[i] = polynomial_arithmetic::evaluate(poly.data(), x[i], n);
        }
        std::array<FF, n> dest;
        std::copy(src.begin(), src.end(), dest.begin());

        polynomial_arithmetic::compute_efficient_interpolation(dest.data(), dest.data(), x.data(), n);

        bool correct = true;
        for (size_t i = 0; i < n; i++) {
            if (dest[i] != poly[i]) {
                correct = false;
                break;
            }
```

```
        }
        std::cout << "P-07 Case 1 (distinct points): interpolation correct = " << correct << std::endl;
        EXPECT_TRUE(correct);
    }

    // Case 2: Duplicate points - should fail but silently produces wrong result
    {
        std::array<FF, n> x = { FF(1), FF(2), FF(2) };  // duplicate!
        std::array<FF, n> src;
        for (size_t i = 0; i < n; i++) {
            src[i] = polynomial_arithmetic::evaluate(poly.data(), x[i], n);
        }
        std::array<FF, n> dest;
        std::copy(src.begin(), src.end(), dest.begin());

        // This should ideally throw or assert, but it silently produces wrong output
        polynomial_arithmetic::compute_efficient_interpolation(dest.data(), dest.data(), x.data(), n);

        bool correct = true;
        for (size_t i = 0; i < n; i++) {
            if (dest[i] != poly[i]) {
                correct = false;
                break;
            }
        }
        std::cout << "P-07 Case 2 (duplicate points): interpolation correct = " << correct << std::endl;
        if (!correct) {
            std::cout << "P-07 CONFIRMED: duplicate points produce wrong interpolation with no error"
            ↪  << std::endl;
        }
        // We expect this to be WRONG - the interpolation is ill-defined for duplicate points
        // The test documents the silent failure behavior
    }
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P07*"
```

Recommended Mitigation: Add a distinct-points assertion at the start of `compute_efficient_interpolation`.

```
for (size_t i = 0; i < n; ++i)
    for (size_t j = i + 1; j < n; ++j)
        BB_ASSERT(evaluation_points[i] != evaluation_points[j]);
```

O(n²) but not in the hot path — the only production caller is the `Polynomial` interpolation constructor, called once during polynomial construction

Aztec: Fixed in 12f6f61.

Cyfrin: Verified.

7.1.3 `evaluate_mle` crashes (SIGSEGV) on single-coefficient polynomial

Description: In `polynomial.hpp:399`, `_evaluate_mle` unconditionally accesses `evaluation_points[0]` without checking if the span is empty:

```
Fr_ _evaluate_mle(std::span<const Fr_> evaluation_points,
                  const SharedShiftedVirtualZeroesArray<Fr_>& coefficients,
                  bool shift)
{
    if (coefficients.size() == 0) {
        return Fr_(0);
```

```
    }
    const size_t n = evaluation_points.size();                          // n = 0
    const size_t dim = numeric::get_msb(coefficients.end_ - 1) + 1;  // dim = get_msb(0) + 1 = 1
    // ...
    size_t n_l = 1 << (dim - 1);                                        // n_l = 1
    // ...
    Fr_ u_l = evaluation_points[0];   // OOB: span is empty!
```

For a `Polynomial(1)` (single coefficient, virtual_size=1), calling `evaluate_mle({})` with an empty evaluation point vector causes:

1. `n = 0` (no evaluation points)
2. `dim = get_msb(0) + 1 = 1` (from `coefficients.end_ = 1`)
3. `n_l = 1 << 0 = 1` (one iteration expected)
4. `evaluation_points[0]` -- out-of-bounds read on empty span, **SIGSEGV**

A single-coefficient polynomial represents a constant (a 0-variable multilinear polynomial). Evaluating it at an empty point set should return the constant value.

Impact: Low. `evaluate_mle` is a public API on `Polynomial` that crashes (SIGSEGV) on valid input — a single-coefficient polynomial with an empty evaluation point vector. In practice, sumcheck and other MLE consumers always work with polynomials of size 2, so `evaluation_points` is never empty. But the function has no guard, and a caller constructing a single-element polynomial would hit undefined behavior with no indication of the cause.

Proof of Concept: Integrated test `AuditPoC_P27_EvaluateMleSingleCoefficientCrash` in `polynomial_arithmetic.test.cpp`:

```
TYPED_TEST(PolynomialTests, AuditPoC_P27_EvaluateMleSingleCoefficientCrash)
{
    using FF = TypeParam;

    Polynomial<FF> poly(1);
    poly.at(0) = FF(42);

    // 0-variable MLE: empty evaluation points, should return the constant 42
    std::vector<FF> u; // empty
    // This crashes with SIGSEGV - _evaluate_mle accesses evaluation_points[0] on empty span
    FF result = poly.evaluate_mle(u);
    EXPECT_EQ(result, FF(42));
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P27*"
```

```
[  DEATH   ] Exit code 139 (SIGSEGV)
```

Crashes with SIGSEGV — `_evaluate_mle` accesses `evaluation_points[0]` on an empty span.

Recommended Mitigation: Add after `polynomial.hpp:411`:

```
  const size_t n = evaluation_points.size();
+ if (n == 0) {
+     return coefficients.size() > 0 ? coefficients.get(0) : Fr_(0);
+ }
  const size_t dim = numeric::get_msb(coefficients.end_ - 1) + 1;
```

Aztec: Fixed in 780139e.

Cyfrin: Verified.

7.1.4 Interpolation constructor does not assert equal sizes of interpolation points and evaluations

Description: The interpolation constructor accepts two `std::span` arguments, `interpolation_points` and `evaluations`, and uses the size of `interpolation_points` to determine the polynomial size. It then passes both raw data pointers along with this size to `compute_efficient_interpolation`. There is no assertion or check that `evaluations.size()` is at least as large as `interpolation_points.size()`. If the evaluations span is shorter, the interpolation function reads past the end of the evaluations buffer.

```
// polynomial.cpp, lines 115-125
template <typename Fr>
Polynomial<Fr>::Polynomial(std::span<const Fr> interpolation_points,
                           std::span<const Fr> evaluations,
                           size_t virtual_size)
    : Polynomial(interpolation_points.size(), virtual_size)
{
    BB_ASSERT_GT(coefficients_.size(), static_cast<size_t>(0));

    // evaluations.size() could be < interpolation_points.size() -- no check!
    polynomial_arithmetic::compute_efficient_interpolation(
        evaluations.data(), coefficients_.data(), interpolation_points.data(), coefficients_.size());
}
```

Inside `compute_efficient_interpolation`, the `n` parameter (set to `coefficients_.size()`, which equals `interpolation_points.size()`) is used to index into `src` (the evaluations data pointer) up to index `n-1`:

```
// polynomial_arithmetic.cpp, lines 289-292
for (size_t i = 0; i < n; ++i) {
    roots_and_denominators[i] = -evaluation_points[i];
    temp_src[i] = src[i];     // reads evaluations[i] -- OOB if i >= evaluations.size()
    dest[i] = 0;
```

Impact: Out-of-bounds read on the evaluations buffer. The interpolation produces incorrect polynomial coefficients based on whatever data lies beyond the evaluations array.

Current call sites pass matched interpolation/evaluation spans. A mismatched internal call would produce incorrect local proof construction; there is no evidence this would make a verifier accept an invalid proof.

Proof of Concept:

```
TEST(LowFindings, ECA_02_InterpolationConstructorMissingSizeCheck)
{
    // Backing buffer of 4 evaluations - but we only intend to pass the first 2.
    std::vector<FF> all_evaluations = { FF(10), FF(20), FF(30), FF(40) };
    std::vector<FF> points = { FF(1), FF(2), FF(3), FF(4) };

    // The "evaluations" span we hand to the constructor is shorter (size 2) than
    // the interpolation points span (size 4).
    std::span<const FF> short_evals(all_evaluations.data(), 2);

    // The constructor uses interpolation_points.size() (=4) as the loop bound
    // inside compute_efficient_interpolation, so it reads evaluations[0..3] from
    // the underlying buffer - past the end of the span we passed.
    auto poly = bb::Polynomial<FF>(points, short_evals, /*virtual_size=*/4);

    // The reconstructed polynomial reproduces ALL FOUR backing values, including
    // the two that lie BEYOND the supplied span. With a proper size assertion
    // those reads should never have happened.
    EXPECT_EQ(poly.evaluate(FF(1)), FF(10));
    EXPECT_EQ(poly.evaluate(FF(2)), FF(20));
    EXPECT_EQ(poly.evaluate(FF(3)), FF(30)); // <-- read PAST evaluations.size()
    EXPECT_EQ(poly.evaluate(FF(4)), FF(40)); // <-- read PAST evaluations.size()
}
```

Recommended Mitigation: Add a size assertion at the top of the constructor body:

```
BB_ASSERT_EQ(interpolation_points.size(), evaluations.size());
```

Aztec: Fixed in 80131f6.

Cyfrin: Verified.

7.1.5 `get_scratch_space` shared buffer unsafe under concurrent FFTs

Description: In `polynomial_arithmetic.cpp:21`, `get_scratch_space` returns a `shared_ptr` to a **single static buffer** reused across sequential calls to avoid repeated allocation. The mutex protects allocation but does **not** prevent two concurrent threads from receiving the same buffer and writing to it simultaneously.

```
template <typename Fr> std::shared_ptr<Fr[]> get_scratch_space(const size_t num_elements)
{
    static std::mutex scratch_mutex;
    std::lock_guard lock(scratch_mutex);          // only protects allocation
    static std::shared_ptr<Fr[]> working_memory = nullptr;
    static size_t current_size = 0;
    if (num_elements > current_size) {
        working_memory = std::make_shared<Fr[]>(num_elements);
        current_size = num_elements;
    }
    return working_memory;                        // returns same pointer to all callers
}
```

The lock serializes the allocation but not the **usage**. Both threads get a `shared_ptr` to the same underlying array and write butterfly results into overlapping indices, corrupting each other's output.

Note: PR22306 (20392b77aa) added this `std::mutex` to `get_scratch_space` to protect the allocation logic. However, this only serializes the check-and-allocate step — the **concurrent usage** issue (two callers receiving the same buffer) remains unfixed.

Impact: Low. This is a reproducible data corruption with PoC (50/50 trials corrupted). In barretenberg's current architecture, FFTs are not called concurrently — `parallel_for` parallelizes within a single FFT. The shared scratch buffer is safe under this assumption, but the code does not enforce it and the mutex gives a false sense of thread safety.

Proof of Concept: `AuditPoC_P30_ScratchSpaceConcurrentCorruption` in `polynomial_arithmetic.test.cpp`:

```
TEST(AuditPoC, P30_ScratchSpaceConcurrentCorruption)
{
    using FF = bb::fr;
    constexpr size_t n = 256;

    std::vector<FF> roots_a(n), roots_b(n);
    for (size_t i = 0; i < n; i++) {
        roots_a[i] = FF(i + 1);
        roots_b[i] = FF(i + 1000);
    }
    std::vector<FF> dest_a(n + 1), dest_b(n + 1);
    std::vector<FF> ref_a(n + 1), ref_b(n + 1);
    polynomial_arithmetic::compute_linear_polynomial_product(roots_a.data(), ref_a.data(), n);
    polynomial_arithmetic::compute_linear_polynomial_product(roots_b.data(), ref_b.data(), n);

    constexpr size_t num_trials = 50;
    size_t corruption_count = 0;
    for (size_t trial = 0; trial < num_trials; trial++) {
        std::atomic<bool> go{ false };
        std::thread thread_a([&]() { while (!go.load()) {}
        ↪  polynomial_arithmetic::compute_linear_polynomial_product(roots_a.data(), dest_a.data(), n);
        ↪  });
```

```
        std::thread thread_b([&]() { while (!go.load()) {}
        ↪  polynomial_arithmetic::compute_linear_polynomial_product(roots_b.data(), dest_b.data(), n);
        ↪  });
        go.store(true);
        thread_a.join();
        thread_b.join();
        bool ok = true;
        for (size_t i = 0; i <= n; i++) { if (dest_a[i] != ref_a[i] || dest_b[i] != ref_b[i]) ok =
        ↪  false; }
        if (!ok) corruption_count++;
    }
    // corruption_count == 50/50
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*P30*"
```

```
P-30: 50/50 trials had corrupted results
P-30 CONFIRMED: concurrent scratch space usage causes data corruption
```

Parallelization Opportunities Currently Blocked

`get_scratch_space` is reachable only via the call chain:

```
get_scratch_space
   compute_linear_polynomial_product       (only caller)
         compute_efficient_interpolation   (only caller)
               Polynomial(span, span, vsize) interpolation constructor
               SmallSubgroupIPAProver::compute_eccvm_challenge_polynomial
               ↪  (small_subgroup_ipa.cpp:226)
               SmallSubgroupIPAProver::compute_monomial_coefficients
               ↪  (small_subgroup_ipa.cpp:436)
               ZKSumcheckData::compute_concatenated_libra_polynomial         (zk_sumcheck_data.hpp:230)
```

All four reachable sites currently run sequentially because the shared scratch buffer makes concurrent invocation unsafe. Once this finding is fixed, the following call sites become legally parallelizable:

Site
small_subgroup_ipa.cpp:336-357 — `compute_lagrange_first_and_last` makes two independent `compute_monomial_coefi`
`SmallSubgroupIPAProver::prove()` over multiple proofs (Chonk / folding pipelines, batched circuits)
Future Lagrange-batched routines (e.g., constructing several `Polynomial(points, evals_i, vsize)` inside a `parallel_for`)

Sites NOT blocked (for completeness):

- Gemini's `compute_fold_polynomials` (`gemini_impl.hpp:124-157`) — has a hard data dependency across folds (A_{l+1} depends on A_l). Already uses `parallel_for_heuristic` *within* each fold, which is the only parallelism that algorithm admits.
- Shplonk's per-claim quotient batching (`shplonk.hpp:75-117`) — uses `factor_roots`, not `compute_efficient_interpolation`. Independent per claim and could be parallelized as a separate optimization, but isn't blocked by this finding.

The largest practical payoff is the second row above: enabling Chonk/folding pipelines to safely invoke independent `SmallSubgroupIPAProver` instances concurrently. The mutex inside `get_scratch_space` even gives reviewers a *false signal* of thread safety, so the fix also removes a long-term tripwire — the natural "wrap a `compute_efficient_interpolation` loop in `parallel_for`" refactor would silently corrupt proofs today.

Recommended Mitigation: Give each thread its own buffer with `thread_local`:

```
template <typename Fr> std::shared_ptr<Fr[]> get_scratch_space(const size_t num_elements)
{
    thread_local std::shared_ptr<Fr[]> working_memory = nullptr;
    thread_local size_t current_size = 0;
    if (num_elements > current_size) {
        working_memory = std::make_shared<Fr[]>(num_elements);
        current_size = num_elements;
    }
    return working_memory;
}
```

Note: a deeper fix that removes `get_scratch_space` entirely — by rewriting `compute_linear_polynomial_product` in place — is tracked separately as issue-21. That rewrite would also close the concurrency concern here as a side effect.

Aztec: Fixed in 044b745.

Cyfrin: Verified.

7.1.6 `parse_size_string` does not check for overflow in `value * multiplier`

Description: `parse_size_string()` in `backing_memory.cpp` parses a human-readable size string (e.g., `"500m"`, `"2g"`) into a byte count. After extracting the numeric value via `std::stoull` and selecting the appropriate multiplier based on the suffix, it returns the product `value * multiplier` without checking whether that multiplication overflows `size_t`. On a 64-bit system, `size_t` is 8 bytes, and `value * multiplier` silently wraps modulo 2^64.

```
// backing_memory.cpp, lines 33-73
size_t parse_size_string(const std::string& size_str)
{
    if (size_str.empty()) {
        return std::numeric_limits<size_t>::max();
    }

    try {
        std::string str = size_str;
        char suffix = static_cast<char>(std::tolower(static_cast<unsigned char>(str.back())));
        size_t multiplier = 1;

        if (suffix == 'k') {
            multiplier = 1024ULL;
            str.pop_back();
        } else if (suffix == 'm') {
            multiplier = 1024ULL * 1024ULL;
            str.pop_back();
        } else if (suffix == 'g') {
            multiplier = 1024ULL * 1024ULL * 1024ULL;
            str.pop_back();
        } else if (std::isdigit(static_cast<unsigned char>(suffix)) == 0) {
            throw_or_abort("Invalid storage size format: ...");
        }

        if (str.empty()) {
            throw_or_abort("Invalid storage size format: ...");
        }

        size_t value = std::stoull(str);
        return value * multiplier;   // <--- unchecked overflow
    } catch (...) { ... }
}
```

For example, `"18014398509481984k"` parses as `value = 2^54` and `multiplier = 1024 = 2^10`. The product is

`2^64`, which wraps to `0`. The storage budget is then set to `0`, causing all subsequent file-backed allocations to fail the budget check and silently fall back to RAM.

Impact: An operator specifying a large `BB_STORAGE_BUDGET` env var value gets a silently tiny budget. All file-backed allocations fail, falling back to RAM. There is no memory safety issue; the consequence is unexpected performance behavior (RAM instead of mmap), not corruption. The env var is set by trusted operators, not attacker-controlled.

This is operator-controlled configuration. It is suitable as a Low severity reliability issue, not an externally exploitable security issue.

Proof of Concept: At the shell level:

```
# Operator intends to set a very large budget:
BB_SLOW_LOW_MEMORY=1 BB_STORAGE_BUDGET="18014398509481984k" ./bb prove ...
# Internally: 1801439850948l984 * 1024 = 2^64, wraps to 0. storage_budget = 0
# In low-memory mode, any required_bytes > 0 causes the file-backed admission
# check to fail, silently falling back to aligned heap memory.
```

```
TEST(LowFindings, ECA_04_ParseSizeStringOverflow)
{
    // Sanity: well-formed inputs parse correctly.
    EXPECT_EQ(parse_size_string("1k"), 1024u);
    EXPECT_EQ(parse_size_string("2g"), 2ULL * 1024 * 1024 * 1024);

    // 2^54 * 1024 == 2^64 → wraps to 0 silently. An operator who set
    // BB_SLOW_LOW_MEMORY=1 BB_STORAGE_BUDGET=18014398509481984k would actually
    // get a budget of 0 bytes, disabling all file-backed allocation.
    EXPECT_EQ(parse_size_string("18014398509481984k"), 0u);
}
```

Recommended Mitigation: Add an overflow guard before the multiplication:

```
size_t value = std::stoull(str);
if (value > std::numeric_limits<size_t>::max() / multiplier) {
    throw_or_abort("Storage size overflows: '" + size_str + "'");
}
return value * multiplier;
```

Aztec: Fixed in 87513f8.

Cyfrin: Verified.

7.1.7 `EvaluationDomain` copy constructor off-by-one for size=2 domains

Description: In `evaluation_domain.cpp:112`, the copy constructor allocates `round_roots` using the formula `log2_size - 1`:

```
round_roots.resize(log2_size - 1);              // for log2_size=1: resize(0)
inverse_round_roots.resize(log2_size - 1);
round_roots[0] = &roots[0];                     // OOB write when resize(0)!
inverse_round_roots[0] = &roots.get()[size];
for (size_t i = 1; i < log2_size - 1; ++i) {
    round_roots[i] = round_roots[i - 1] + (1UL << i);
    inverse_round_roots[i] = inverse_round_roots[i - 1] + (1UL << i);
}
```

For a size-2 domain ($\log_2(2) = 1$), this resizes to 0 elements, then writes to `round_roots[0]` — an out-of-bounds write on an empty vector (undefined behavior).

The original `compute_lookup_table_single` correctly uses `emplace_back` which always adds at least 1 entry unconditionally, then grows dynamically. But the copy constructor pre-allocates with the formula `log2_size - 1`, which underestimates by 1 when `log2_size = 1`:

- `log2_size = 1`: original creates 1 entry, copy constructor creates 0 entries (bug)
- `log2_size = 2`: both create 1 entry
- `log2_size = 3`: both create 2 entries

Impact: Low. Copying an `EvaluationDomain<Fr>` of size 2 after `compute_lookup_table()` has been called causes undefined behavior. In practice, FFT domains are typically larger (2^10+), but size-2 domains are valid and could appear.

Proof of Concept: Integrated test `AuditPoC_P22_EvalDomainCopyCtorOffByOne` in `polynomial_arithmetic.test.cpp`:

```
TYPED_TEST(PolynomialTests, AuditPoC_P22_EvalDomainCopyCtorOffByOne)
{
    using FF = TypeParam;

    auto domain = EvaluationDomain<FF>(2);
    domain.compute_lookup_table();

    // Copy the domain - triggers the off-by-one bug:
    // round_roots.resize(log2_size - 1) = resize(0), then writes round_roots[0]
    // This is UB (out-of-bounds write on empty vector)
    auto copied = EvaluationDomain<FF>(domain);
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P22*"
```

```
[==========] Running 2 tests from 2 test suites.
[ RUN      ] PolynomialTests/0.AuditPoC_P22_EvalDomainCopyCtorOffByOne
[1]    48511 segmentation fault  ./bin/polynomials_tests --gtest_filter="*AuditPoC_P22*"
```

The test crashes with a segfault, confirming the OOB write on the empty `round_roots` vector.

The formula `log2_size - 1` should be `log2_size` (or `std::max(1UL, log2_size - 1)`) to handle the size=2 case correctly.

Recommended Mitigation: In `evaluation_domain.cpp:112-113`:

```
-        round_roots.resize(log2_size - 1);
-        inverse_round_roots.resize(log2_size - 1);
+        round_roots.resize(std::max(size_t(1), log2_size - 1));
+        inverse_round_roots.resize(std::max(size_t(1), log2_size - 1));
```

Aztec: Fixed in 6cd21a0.

Cyfrin: Verified.

7.1.8 `UnivariateCoefficientBasis operator==` compares unused `coefficients[2]` for `domain_end=2`

Description: In `univariate_coefficient_basis.hpp:87`, `operator==` is defaulted:

```
bool operator==(const UnivariateCoefficientBasis& other) const = default;
```

The compiler-generated `= default` compares **all** members, including all 3 elements of `std::array<Fr, 3> coefficients` — even `coefficients[2]`.

For `(domain_end=2, has_a0_plus_a1=false)` objects, `coefficients[2]` is semantically unused (neither an x^2 coefficient nor a valid Karatsuba precomputation). However, it may contain different values depending on the construction path, causing `operator==` to return `false` for two objects that represent the same polynomial.

Construction paths that leave `coefficients[2]` with different values:

1. Cross-`has_a0_plus_a1` constructor (line 59): Copies `[0]` and `[1]` from a `(2, true)` source, does NOT set `[2]` — it is left uninitialized:

```
UnivariateCoefficientBasis(const UnivariateCoefficientBasis<Fr, domain_end, true>& other)
    requires(!has_a0_plus_a1)
{
    coefficients[0] = other.coefficients[0];
    coefficients[1] = other.coefficients[1];
    // coefficients[2] NOT set for domain_end=2 → uninitialized
    if constexpr (domain_end == 3) {
        coefficients[2] = other.coefficients[2];
    }
}
```

2. Default constructor (line 57) (`= default`): Leaves the entire array uninitialized for trivial types.
3. Arithmetic operators (`operator+`, `operator-`, etc.): Copy `*this` into `res` (copying whatever `[2]` held), then modify only `[0]` and `[1]`.

During sumcheck, `UnivariateCoefficientBasis<Fr, 2, true>` objects are naturally constructed from Lagrange-basis polynomials with `[2] = a0 + a1` precomputed for Karatsuba. After arithmetic, results become `(2, false)`. Two objects representing the same polynomial can end up with different `[2]` values depending on the construction path:

- **Path A** — cross-constructor `(2, true)` → `(2, false)`: copies `[0]` and `[1]`, leaves `[2]` uninitialized
- **Path B** — cross-constructor then `operator+`: `res(*this)` copies all 3 elements including `[2]`, then only updates `[0]` and `[1]`

Both produce the same polynomial, but `[2]` differs → `operator==` returns false.

Impact: Low. `operator==` produces false negatives — two objects representing the same polynomial compare as not equal. The `operator==` is currently not called in production code, but `UnivariateCoefficientBasis` is core sumcheck machinery, and equality comparison is a natural operation to add.

The `(2, true)` case is not affected (`coefficients[2] = a + a` is deterministic). The `(3, false)` case is also fine (`[2]` is the actual x^2 coefficient).

Proof of Concept: `AuditPoC_P35_EqualityComparesUnusedCoefficient` — builds two `(2, false)` objects with identical `[0]` and `[1]` but different `[2]`, then shows `operator==` returns false:

```
TYPED_TEST(UnivariateCoefficientBasisTest, AuditPoC_P35_EqualityComparesUnusedCoefficient)
{
    // Start from a (2, true) object - as naturally constructed from Univariate (Lagrange basis)
    // P(X) = 5 + 3X, so a0=5, a1=3, a0+a1=8
    UnivariateCoefficientBasis<fr, 2, true> src;
    src.coefficients[0] = fr(5);
    src.coefficients[1] = fr(3);
    src.coefficients[2] = fr(8); // a0 + a1 (Karatsuba precomputation)

    // Path A: cross-constructor (2, true) → (2, false)
    // copies [0] and [1], but does NOT set [2] → uninitialized
    UnivariateCoefficientBasis<fr, 2, false> a(src);

    // Path B: cross-constructor then arithmetic (operator+ copies [2] from *this)
    UnivariateCoefficientBasis<fr, 2, false> b(src);
    UnivariateCoefficientBasis<fr, 2, false> zero;
    zero.coefficients[0] = fr(0);
    zero.coefficients[1] = fr(0);
    b = b + zero; // operator+ copies b's [2] into result via res(*this)

    // Both represent P(X) = 5 + 3X, but [2] may differ
    // BUG: operator== (= default) compares unused [2]
    EXPECT_NE(a, b);
```

```
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*P35*"
```

Recommended Mitigation: In univariate_coefficient_basis.hpp:87, replace `= default` with a custom operator== that only compares semantically meaningful coefficients:

```
-    bool operator==(const UnivariateCoefficientBasis& other) const = default;
+    bool operator==(const UnivariateCoefficientBasis& other) const
+    {
+        bool eq = (coefficients[0] == other.coefficients[0]) &&
+                  (coefficients[1] == other.coefficients[1]);
+        if constexpr (domain_end == 3 || has_a0_plus_a1) {
+            eq = eq && (coefficients[2] == other.coefficients[2]);
+        }
+        return eq;
+    }
```

Aztec: Fixed in 80131f6.

Cyfrin: Verified

7.1.9 Signed integer overflow in `_evaluate_mle` bit-shift expressions

Description: The `_evaluate_mle` function in `polynomial.hpp` uses the integer literal `1` (type `int`, 32-bit signed) as the left operand of the left-shift operator << in three places:

```
// polynomial.hpp:415
BB_ASSERT_EQ(coefficients.virtual_size(), static_cast<size_t>(1 << n));

// polynomial.hpp:421
size_t n_l = 1 << (dim - 1);

// polynomial.hpp:448
n_l = 1 << (dim - l - 1);
```

Per the C++ standard ([expr.shift]/1), shifting a signed integer by an amount greater than or equal to its bit-width, or shifting into the sign bit, is undefined behavior. When `n >= 31`, the expression `1 << n` overflows a 32-bit signed `int`. Concretely:

- `1 << 31` produces `INT_MIN` (-2147483648) in two's complement.
- `static_cast<size_t>(INT_MIN)` then sign-extends to `0xFFFFFFFF80000000` on a 64-bit system, which is NOT `2^31`.
- The assertion on line 415 spuriously fails, or worse, a conforming compiler is entitled to optimize away the entire check because the expression involves UB.

The same issue affects the buffer size computation on line 421 and the loop variable on line 448. If `dim >= 32`, these produce garbage values that control loop bounds and allocation sizes.

The codebase already uses the correct `1UL << d` pattern elsewhere, for instance in `eq_polynomial.hpp`:

```
// eq_polynomial.hpp:145 - correct pattern
const size_t N = 1UL << d;
```

The three occurrences in `_evaluate_mle` are inconsistent with this established convention.

Impact: For circuits with 2^31 or more rows, MLE evaluation silently produces incorrect results or crashes. The current maximum circuit size is `CONST_SIZE_PROOF_LOG_N = 28`, so this is not reachable today. However, the trend in proving systems is toward larger circuits, and this is actual C++ undefined behavior — the compiler is not

required to produce any particular result, and optimizations based on UB assumptions can silently break seemingly unrelated code.

Recommended Mitigation: Replace the signed `1` literal with `size_t(1)` in all three locations:

```
// Line 415
BB_ASSERT_EQ(coefficients.virtual_size(), size_t(1) << n);

// Line 421
size_t n_l = size_t(1) << (dim - 1);

// Line 448
n_l = size_t(1) << (dim - l - 1);
```

Aztec: Fixed in 87513f8.

Cyfrin: Verified.

7.1.10 `EvaluationDomain` move-assignment self-assignment permanently destroys lookup tables

Description: The `EvaluationDomain<Fr>::operator=(EvaluationDomain&&)` move-assignment operator in `evaluation_domain.cpp` lacks a self-assignment guard. When `domain = std::move(domain)` is executed, the function first destroys its own state and then attempts to read from `other` — which is the same object:

```
// evaluation_domain.cpp:147-172
template <typename Fr> EvaluationDomain<Fr>& EvaluationDomain<Fr>::operator=(EvaluationDomain&& other)
{
    size = other.size;
    generator_size = other.generator_size;
    // ... scalar copies work fine since this == &other ...

    roots = nullptr;                        // (1) destroys this->roots, which IS other.roots
    round_roots.clear();                    // (2) destroys this->round_roots, which IS other.round_roots
    inverse_round_roots.clear();            // (3) destroys this->inverse_round_roots

    if (other.roots != nullptr) {           // (4) always FALSE - we just set it to nullptr at (1)
        roots = other.roots;                //     never reached
        round_roots = std::move(other.round_roots);
        inverse_round_roots = std::move(other.inverse_round_roots);
    }
    other.roots = nullptr;                  // (5) redundant - already nullptr
    return *this;
}
```

The sequence is:

1. Line 162: `roots = nullptr` — since `this == &other`, this also nullifies `other.roots`. If this was the last `shared_ptr` reference, the root-of-unity array is deallocated.
2. Lines 163-164: `round_roots.clear(); inverse_round_roots.clear()` — destroys the pointer vectors for both `this` and `other`.
3. Line 165: `if (other.roots != nullptr)` — evaluates to `false` because step 1 already nullified it.
4. All precomputed FFT lookup table data is permanently lost. Any subsequent FFT/IFFT call will crash or produce garbage.

Both `Polynomial::operator=(const Polynomial&)` and `BackingMemory::operator=(BackingMemory&&)` in the same codebase correctly check for self-assignment:

```
// polynomial.cpp:137-144 - correct pattern
template <typename Fr> Polynomial<Fr>& Polynomial<Fr>::operator=(const Polynomial<Fr>& other)
{
    if (this == &other) {
```

```
        return *this;
    }
    coefficients_ = _clone(other.coefficients_);
    return *this;
}

// backing_memory.hpp:78-89 - correct pattern
BackingMemory& operator=(BackingMemory&& other) noexcept
{
    if (this != &other) {
        raw_data = other.raw_data;
        // ...
    }
    return *this;
}
```

`EvaluationDomain` is the only move-assignment operator in the module that omits this guard.

Impact: Self-move-assignment can occur through generic algorithms (e.g., `container[i] = std::move(container[j])` where `i == j` at runtime), `std::swap` idioms with moved-from temporaries, or standard library operations on containers of `EvaluationDomain` objects. The result is permanent, irrecoverable loss of all precomputed FFT root tables, causing null pointer dereference or incorrect FFT results on subsequent use. Not currently reachable in normal proving flows, but the inconsistency with sibling operators suggests an oversight.

Recommended Mitigation: Add the same self-assignment guard used by `Polynomial` and `BackingMemory`:

```
template <typename Fr>
EvaluationDomain<Fr>& EvaluationDomain<Fr>::operator=(EvaluationDomain&& other)
{
    if (this == &other) {
        return *this;
    }
    // ... rest unchanged
}
```

Aztec: Fixed in 80131f6.

Cyfrin: Verified.

7.1.11 `Polynomial::random(size, start_index)` underflows when `start_index > size`

Description: The two-argument `Polynomial::random` overload computes the allocation size as `size - start_index`. Both parameters are `size_t` (unsigned), so when `start_index > size`, the subtraction wraps to a near-`SIZE_MAX` value:

```
// polynomial.hpp:274-289
static Polynomial random(size_t size, size_t start_index = 0)
{
    BB_BENCH_NAME("generate random polynomial");
    return random(size - start_index, size, start_index);
    //             ^^^^^^^^^^^^^^^^^^  wraps if start_index > size
}

static Polynomial random(size_t size, size_t virtual_size, size_t start_index)
{
    Polynomial p(size, virtual_size, start_index, DontZeroMemory::FLAG);
    parallel_for_heuristic(
        size,
        [&](size_t i) { p.coefficients_.data()[i] = Fr::random_element(); },
        thread_heuristics::ALWAYS_MULTITHREAD);
    return p;
```

```
}
```

The three-argument overload then calls `allocate_backing_memory`, which has its own assertion:

```
BB_ASSERT_LTE(start_index + size, virtual_size);
```

But this assertion is also defeated by unsigned wraparound. For example, `random(4, 10)` computes `size - start_index = 4 - 10 = SIZE_MAX - 5`. Inside `allocate_backing_memory`, the check becomes `10 + (SIZE_MAX - 5) = SIZE_MAX + 5`, which wraps to `4`, and `4 <= 4` passes. The subsequent `BackingMemory::allocate(SIZE_MAX - 5)` attempts a roughly 2^64 byte allocation that fails with `std::bad_alloc`.

The naming is also misleading: in the two-argument form, the `size` parameter actually represents `virtual_size`, not the allocation size. A caller might reasonably write `Polynomial::random(4, /*start_index=*/1)` expecting 4 random elements starting at index 1, but actually gets 3 random elements with `virtual_size=4`.

Impact: OOM crash (not silent corruption). The function is used internally with correct arguments. The underflow is caught by the memory allocator throwing `std::bad_alloc`, so no exploitable memory corruption occurs. The issue is a fragile helper API that turns invalid arguments into an attempted huge allocation instead of rejecting them with a clear assertion at the boundary.

Proof of Concept:

```
TEST(LowFindings, ECA_08_RandomTwoArgUnderflow)
{
    // random(size=4, start_index=10) computes random(4 - 10, 4, 10).
    //   - 4 - 10 underflows in size_t to SIZE_MAX - 5, requested as the alloc size.
    //   - allocate_backing_memory's internal BB_ASSERT_LTE(start + size, virtual)
    //     also wraps (10 + (SIZE_MAX-5) = SIZE_MAX+5 == 4 in size_t), so the
    //     check passes and does NOT catch the bug.
    //   - The allocator then tries to allocate ~2^64 bytes and throws std::bad_alloc.
    EXPECT_THROW(bb::Polynomial<FF>::random(/*size=*/4, /*start_index=*/10), std::bad_alloc);
}
```

Recommended Mitigation: Add a precondition check before the subtraction:

```
static Polynomial random(size_t size, size_t start_index = 0)
{
    BB_ASSERT_GTE(size, start_index);
    return random(size - start_index, size, start_index);
}
```

Aztec: Fixed in 6cd21a0.

Cyfrin: Verified.

7.1.12 `ProverEqPolynomial::construct` returns `[0]` instead of `[1]` for empty challenges

Description: When `ProverEqPolynomial::construct()` is called with an empty challenges vector (`d=0`), the mathematically expected result is the constant polynomial `[1]` (the empty product equals 1). The function first computes the scaling factor:

```
// eq_polynomial.hpp:56-70
static Polynomial<FF> construct(std::span<const FF> challenges, size_t log_num_monomials)
{
    FF scaling_factor = compute_scaling_factor(challenges);
    // For empty challenges: scaling_factor = 1 (empty product), non-zero

    if (scaling_factor.is_zero()) {
        return construct_eq_with_edge_cases(challenges, log_num_monomials);
        // This fallback handles d=0 correctly: loop doesn't execute, returns [1]
    }
```

```
    // Optimal path - delegates to GateSeparatorPolynomial (out of scope)
    return GateSeparatorPolynomial<FF>::compute_beta_products(
        transform_challenge(challenges), log_num_monomials, scaling_factor);
};
```

For empty challenges:

1. `compute_scaling_factor({})` returns `1` (the empty product of `(1 - r_i)` terms).
2. Since `1 != 0`, the optimal path is taken, calling `GateSeparatorPolynomial::compute_beta_products({}, 0, 1)`.
3. This out-of-scope function hits its `betas.empty()` early return, which creates a size-1 polynomial initialized to `0` and ignores the `scaling_factor = 1` argument.

The `compute_scaling_factor` and `transform_challenge` for empty input:

```
// eq_polynomial.hpp:80-88
static FF compute_scaling_factor(std::span<const FF> challenge)
{
    FF out(1);
    const FF one(1);
    for (auto u_i : challenge) {   // empty loop - no iterations
        out *= (one - u_i);
    }
    return out;   // returns 1
}

// eq_polynomial.hpp:101-116
static std::vector<FF> transform_challenge(std::span<const FF> challenges)
{
    std::vector<FF> result;         // empty
    std::vector<FF> denominators;   // empty
    for (const auto& challenge : challenges) { ... }   // empty loop
    FF::batch_invert(denominators);  // no-op on empty vector
    // ...
    return result;   // empty vector
}
```

The fallback path (`construct_eq_with_edge_cases`) would handle `d=0` correctly — its loop doesn't execute and it returns `[1]`. But the optimal path is chosen because `scaling_factor = 1` is non-zero, and the downstream function discards the scaling factor.

Impact: Only affects `d=0`, which is a degenerate case. Sumcheck requires at least `d=1`, so this is not reachable in normal protocol execution. The bug is technically in `GateSeparatorPolynomial::compute_beta_products` (out of scope), but manifests through the `ProverEqPolynomial` public interface. It is an incorrect public helper result for the mathematically valid empty-product case.

Proof of Concept:

```
TEST(LowFindings, ECA_10_ProverEqPolynomialEmptyChallengesReturnsZero)
{
    std::vector<FF> empty_challenges;
    auto result = bb::ProverEqPolynomial<FF>::construct(empty_challenges, /*log_num_monomials=*/0);

    ASSERT_EQ(result.size(), 1u);

    // Mathematically eq(X, r) with d=0 is the empty product, which equals 1.
    // BUG: GateSeparatorPolynomial::compute_beta_products({}, 0, scaling_factor=1)
    // hits its betas.empty() early return and produces Polynomial(1)=[0],
    // ignoring the scaling_factor. So the public helper returns [0] instead of [1].
    EXPECT_EQ(result.get(0), FF(0));
    // Expected after fix:
```

```
    //   EXPECT_EQ(result.get(0), FF(1));
}
```

Recommended Mitigation: Add a guard at the top of `construct`:

```
static Polynomial<FF> construct(std::span<const FF> challenges, size_t log_num_monomials)
{
    if (challenges.empty()) {
        Polynomial<FF> result(1, 1);
        result.at(0) = FF(1);
        return result;
    }
    // ... rest unchanged
}
```

Alternatively, fix `GateSeparatorPolynomial::compute_beta_products` to respect the `scaling_factor` argument when `betas` is empty.

Aztec: Fixed in 87513f8.

Cyfrin: Verified.

7.1.13 `EvaluationDomain` move constructor leaves inconsistent state

Description: The move constructor nullifies `other.roots` and moves the `round_roots` / `inverse_round_roots` vectors, but leaves `other.size`, `other.root`, `other.domain`, and the other scalar fields at their original nonzero values.

After move, the moved-from domain has `size > 0` (appears valid) but `roots == nullptr` and `round_roots` empty. Any subsequent FFT operation on the moved-from object dereferences null or accesses empty vectors.

Same issue in the move assignment operator.

Impact: Low. In C++ move semantics, using a moved-from object is generally discouraged. But the partial invalidation (`size` still > 0) makes accidental reuse more dangerous than a fully-zeroed state — the object looks valid but crashes on use.

Proof of Concept: Integrated test `AuditPoC_P18_EvalDomainMoveInconsistent` in `polynomials/polynomial_arithmetic.test.cpp`:

```
TEST(polynomials, AuditPoC_P18_EvalDomainMoveInconsistent)
{
    using FF = bb::fr;

    constexpr size_t n = 16;
    auto domain = EvaluationDomain<FF>(n);
    domain.compute_lookup_table();

    // Save original values
    FF original_root = domain.root;

    // Move-construct a new domain
    auto moved = EvaluationDomain<FF>(std::move(domain));

    // moved should have the data
    EXPECT_EQ(moved.size, n);
    EXPECT_EQ(moved.root, original_root);
    EXPECT_NE(moved.get_round_roots().size(), 0UL);

    // Source domain: roots were moved out (nullptr), but size and root are NOT zeroed
    // This is inconsistent - size > 0 suggests a valid domain, but roots == nullptr
    std::cout << "P-18: After move, source domain.size = " << domain.size << std::endl;
    std::cout << "P-18: After move, source domain.root = " << domain.root << std::endl;
```

```
    std::cout << "P-18: After move, source domain round_roots count = "
              << domain.get_round_roots().size() << std::endl;

    bool inconsistent = (domain.size > 0) && (domain.get_round_roots().empty());
    if (inconsistent) {
        std::cout << "P-18 CONFIRMED: moved-from domain has size=" << domain.size
                  << " but empty round_roots" << std::endl;
    }
    EXPECT_TRUE(inconsistent);
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P18*"
```

```
P-18: After move, source domain.size = 16
P-18: After move, source domain.root = ...
P-18: After move, source domain round_roots count = 0
P-18 CONFIRMED: moved-from domain has size=16 but empty round_roots
```

Recommended Mitigation: Make the moved-from object match the **default-constructed empty state** — i.e., restore the invariant `size > 0 → roots != nullptr`. Use `std::exchange` to both steal the field's value and zero the source in one expression:

```
 EvaluationDomain<Fr>::EvaluationDomain(EvaluationDomain&& other)
-    : size(other.size)
-    , num_threads(compute_num_threads(other.size))
-    , thread_size(other.size / num_threads)
-    , log2_size(static_cast<size_t>(numeric::get_msb(size)))
-    , log2_thread_size(static_cast<size_t>(numeric::get_msb(thread_size)))
-    , log2_num_threads(static_cast<size_t>(numeric::get_msb(num_threads)))
-    , generator_size(other.generator_size)
+    : size(std::exchange(other.size, 0))
+    , num_threads(std::exchange(other.num_threads, 0))
+    , thread_size(std::exchange(other.thread_size, 0))
+    , log2_size(std::exchange(other.log2_size, 0))
+    , log2_thread_size(std::exchange(other.log2_thread_size, 0))
+    , log2_num_threads(std::exchange(other.log2_num_threads, 0))
+    , generator_size(std::exchange(other.generator_size, 0))
     , root(other.root)
     , root_inverse(other.root_inverse)
-    , domain(Fr{ size, 0, 0, 0 }.to_montgomery_form())
-    , domain_inverse(domain.invert())
+    , domain(other.domain)
+    , domain_inverse(other.domain_inverse)
     , generator(other.generator)
     , generator_inverse(other.generator_inverse)
+    , roots(std::move(other.roots))            // shared_ptr move → other.roots = nullptr
+    , round_roots(std::move(other.round_roots))
+    , inverse_round_roots(std::move(other.inverse_round_roots))
 {
-    roots = other.roots;
-    round_roots = std::move(other.round_roots);
-    inverse_round_roots = std::move(other.inverse_round_roots);
-    // @audit move must not destruct the source
-    other.roots = nullptr;
 }
```

Apply the same pattern to the move assignment operator.

Reference pattern: `BackingMemory::operator=(BackingMemory&&)` already does this correctly — it nulls `other.raw_data` after stealing its target to preserve the invariant that `raw_data` mirrors the owning pointer. `EvaluationDomain` should mirror this discipline.

Why `std::exchange` is applied to these specific fields — two tiers:

- Must zero (`size`) — this is the only invariant-gating field. All validity checks on an `EvaluationDomain` gate on `size > 0`; zeroing it on move makes the source visibly empty and prevents any downstream code from treating the moved-from object as usable.
- Should zero for internal consistency (`num_threads`, `thread_size`, `log2_size`, `log2_thread_size`, `log2_num_threads`, `generator_size`) — these are derived from `size`. Setting `size = 0` but leaving `log2_size = 4` would be internally incoherent even though no code reads them without first checking `size`. Cheap to fix, keeps the moved-from state coherent.

The field elements (`root`, `domain`, `generator`, etc.) are not invariant-gating, so plain `other.root` is fine — zeroing them is defensive but adds noise. The `shared_ptr` and `vector` members use `std::move`, not `std::exchange`, because moving a `shared_ptr` already nulls the source and moving a `vector` already empties it.

Aztec: Fixed in cf988ed.

Cyfrin: Verified.

7.1.14 `Polynomial` defaulted move leaves inconsistent state

Description: `Polynomial` uses both a defaulted move constructor and move assignment:

```
Polynomial(Polynomial&& other) noexcept = default;                    // line 96
Polynomial& operator=(Polynomial&& other) noexcept = default;         // line 134
```

Both share the same issue.

`Polynomial` has one member, `coefficients_` of type `SharedShiftedVirtualZeroesArray<Fr>`. That struct also has no custom move, so the compiler-generated move performs:

- `start_`, `end_`, `virtual_size_` (`size_t` scalars) — trivially copied, **source unchanged**
- `backing_memory_` (contains `shared_ptr<Fr[]>`, `raw_data`, optional `file_backed`) — moved via `BackingMemory`'s move, which correctly nulls source's pointer fields

The result: after move, the source polynomial has `size() > 0` (scalars unchanged), but `data() == nullptr` (backing memory moved out).

A moved-from `Polynomial` looks valid to most accessors:

- `size()` returns the old non-zero value
- `virtual_size()` returns the old value
- `start_index()` / `end_index()` return old values
- `is_empty()` returns `false`

But any attempt to access backed memory (`at()`, `data()[i]`, `operator+=`, etc.) **dereferences nullptr** → segfault or UB.

Impact: Low. C++ move semantics say moved-from objects should not be used without reassignment. Careful callers avoid this entirely. The danger is that:

1. A caller accidentally uses a moved-from polynomial (common bug class in C++)
2. The polynomial looks valid via scalar queries (size, virtual_size)
3. The crash happens only on data access, with a misleading null-pointer error

Proof of Concept: Integrated death test `AuditPoC_P50_MovedFromPolynomialCausesSegfault` in `polynomials/polynomial.test.cpp`. A caller trusts the scalar fields (`size() > 0`) and writes via `src.at(start_index)`, which dereferences the null `data()` pointer and segfaults — demonstrating that the inconsistent moved-from state (`size() > 0` but `data() == nullptr`) is a real crash path, not just a latent state violation.

```
TEST(PolynomialDeathTest, AuditPoC_P50_MovedFromPolynomialCausesSegfault)
{
    using FF = bb::fr;
    using Polynomial = bb::Polynomial<FF>;

    auto use_moved_from = [] {
        auto src = Polynomial::random(10, 16, 2);
        Polynomial moved{ std::move(src) };

        // After move: src.size() == 10 (looks valid), but src.data() == nullptr.
        src.at(2) = FF(5); // dereferences nullptr
    };

    EXPECT_DEATH(use_moved_from(), ".*");
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P50*"
```

Observed output:

```
[ RUN      ] PolynomialDeathTest.AuditPoC_P50_MovedFromPolynomialCausesSegfault
[       OK ] PolynomialDeathTest.AuditPoC_P50_MovedFromPolynomialCausesSegfault (9 ms)
```

`EXPECT_DEATH` forks a subprocess, runs the write, and confirms the subprocess terminates abnormally (SIGSEGV).

Recommended Mitigation: Fix at the `SharedShiftedVirtualZeroesArray` layer — replace the implicit defaulted move with an explicit one that zeros the scalar fields. This addresses the root cause and propagates up to every wrapper (not just `Polynomial`):

```
SharedShiftedVirtualZeroesArray(SharedShiftedVirtualZeroesArray&& other) noexcept
    : start_(std::exchange(other.start_, 0))
    , end_(std::exchange(other.end_, 0))
    , virtual_size_(std::exchange(other.virtual_size_, 0))
    , backing_memory_(std::move(other.backing_memory_))
{}

SharedShiftedVirtualZeroesArray& operator=(SharedShiftedVirtualZeroesArray&& other) noexcept {
    if (this != &other) {
        start_          = std::exchange(other.start_, 0);
        end_            = std::exchange(other.end_, 0);
        virtual_size_   = std::exchange(other.virtual_size_, 0);
        backing_memory_ = std::move(other.backing_memory_);
    }
    return *this;
}
```

With this fix, the moved-from `Polynomial` has `size() == 0` and `is_empty() == true` — matching the default-constructed empty state. The invariant `size > 0 → data() != nullptr` is restored.

Aztec: Fixed in 6043f43.

Cyfrin: Verified.

7.1.15 `ProverEqPolynomial::construct` fast path does not enforce `challenges.size() == log_num_monomials`

Description: `ProverEqPolynomial::construct()` in `eq_polynomial.hpp` decides between an optimized path (via `GateSeparatorPolynomial::compute_beta_products`) and a fallback (`construct_eq_with_edge_cases`) based on a scaling factor. Only the fallback path asserts the fundamental dimensional precondition `challenges.size() == log_num_monomials` — the fast path has no such check:

```
// eq_polynomial.hpp:56-70
static Polynomial<FF> construct(std::span<const FF> challenges, size_t log_num_monomials)
{
    FF scaling_factor = compute_scaling_factor(challenges);

    if (scaling_factor.is_zero()) {
        return construct_eq_with_edge_cases(challenges, log_num_monomials);
        // construct_eq_with_edge_cases does:
        //   BB_ASSERT_EQ(d, log_num_monomials, "expect log_num_monomials == r.size()");
    }

    // Fast path - no dimension assertion:
    return GateSeparatorPolynomial<FF>::compute_beta_products(
        transform_challenge(challenges), log_num_monomials, scaling_factor);
};
```

Two concrete symptoms follow:

1. `log_num_monomials < challenges.size()` — extra challenges are silently discarded. The returned eq-table is not `eq(X, r)` over the full challenge vector; it corresponds to a smaller-dimensional polynomial. For a weighting object over the Boolean hypercube, dropping a dimension changes the polynomial entirely.
2. `log_num_monomials > challenges.size()` **and no** `r_i == 1` — the fast path continues into `compute_beta_products`, where the inner loop walks `betas[current_element_idx]` past the end of the transformed-challenge vector. This is an out-of-bounds read in the consuming helper, reachable only via this public entry point.

The existing issue 16 (`ProverEqPolynomial::construct returns [0] instead of [1] for empty challenges`) is adjacent but strictly narrower — it addresses only the empty-challenge degenerate case, not the general-dimension mismatch on non-empty input.

Impact: Current production callers pass matched dimensions. A mismatched internal caller either silently computes the wrong eq-polynomial (soundness-relevant if the output feeds into a sumcheck-style relation) or reads out of bounds in the downstream `compute_beta_products`. The fallback path has the right assertion; the fast path should too.

Recommended Mitigation: Hoist the dimension check out of `construct_eq_with_edge_cases` and apply it on both paths:

```
static Polynomial<FF> construct(std::span<const FF> challenges, size_t log_num_monomials)
{
    BB_ASSERT_EQ(challenges.size(), log_num_monomials,
                 "ProverEqPolynomial::construct: challenges.size() must equal log_num_monomials");
    FF scaling_factor = compute_scaling_factor(challenges);
    // ... rest unchanged
}
```

Aztec: Fixed in 80131f6 and 19c03f4.

Cyfrin: Verified.

7.2 Informational

7.2.1 `SharedShiftedVirtualZeroesArray` invariant enforcement gap

Description: In `shared_shifted_virtual_zeroes_array.hpp`, the struct requires the invariant `start_ <= end_ <= virtual_size_`, but this is never validated:

1. No constructor checks the invariant — aggregate initialization allows any values
2. `Polynomial::shrink_end_index` checks the upper bound but not the lower:

```
// polynomial.cpp:236
void Polynomial<Fr>::shrink_end_index(const size_t new_end_index)
{
    BB_ASSERT_LTE(new_end_index, end_index());   // checks: new <= current end
    coefficients_.end_ = new_end_index;            // NO check: new >= start_
}
```

If `new_end_index < start_`, then `size()` (= `end_ - start_`) underflows to `SIZE_MAX` because both are `size_t` (unsigned). This corrupts all size-dependent operations.

Impact: Informational. All current callers use `shrink_end_index` correctly. If misused, the `SIZE_MAX` underflow would crash immediately (not silently produce wrong results). The assert is a code hygiene suggestion.

Proof of Concept: `AuditPoC_P15_ShrinkEndIndexUnderflow` — creates a polynomial with `start_index=2`, then calls `shrink_end_index(1)`. The current check passes (`1 <= 10`) but `end_ < start_` causes `size()` to underflow. With our fix, the assert would catch it.

```
TEST(AuditPoC, P15_ShrinkEndIndexUnderflow)
{
    using FF = bb::fr;

    // Create a polynomial with start_index=2: coefficients at indices [2..9], virtual_size=16
    auto poly = Polynomial<FF>(8, 16, /*start_index=*/2);
    EXPECT_EQ(poly.start_index(), 2UL);
    EXPECT_EQ(poly.end_index(), 10UL);
    EXPECT_EQ(poly.size(), 8UL);

    // shrink_end_index(1): passes current check (1 <= 10) but violates start_ <= end_
    poly.shrink_end_index(1);

    // end_ = 1, start_ = 2 → size() = 1 - 2 = SIZE_MAX (unsigned underflow)
    EXPECT_EQ(poly.end_index(), 1UL);
    EXPECT_EQ(poly.size(), SIZE_MAX);

    // With fix: BB_ASSERT_GTE(new_end_index, start_index()) would catch 1 < 2
}
```

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*P15*"
```

```
P-15: start=2 end=1 size=18446744073709551615
P-15 CONFIRMED: size() underflows to SIZE_MAX
```

Recommended Mitigation: Add lower bound check at `polynomial.cpp:238`:

```
  BB_ASSERT_LTE(new_end_index, end_index());
+ BB_ASSERT_GTE(new_end_index, start_index());
  coefficients_.end_ = new_end_index;
```

Aztec: Fixed in 80131f6.

Cyfrin: Verified.

7.2.2 `set()`, `at()`, and `operator[]` (mutable) use debug-only bounds checks allowing silent heap corruption in release

Description: `SharedShiftedVirtualZeroesArray` provides two classes of accessors. The read-only `get()` method has an explicit runtime bounds check in all builds, while the mutable `set()` and `operator[]` methods guard bounds with `BB_ASSERT_DEBUG`, a macro that compiles to a no-op in release builds (when `NDEBUG` is defined). `Polynomial::at()` delegates directly to `operator[]`, inheriting the same behavior.

The safe read path (`get()`):

```
// shared_shifted_virtual_zeroes_array.hpp, lines 56-64
const T& get(size_t index, size_t virtual_padding = 0) const
{
    static const T zero{};
    BB_ASSERT_DEBUG(index < virtual_size_ + virtual_padding);  // debug-only hint
    if (index >= start_ && index < end_) {                     // ALWAYS runs -- safe
        return data()[index - start_];
    }
    return zero;  // out-of-range returns zero, no memory access
}
```

The unsafe write paths (`set()` and `operator[]`):

```
// shared_shifted_virtual_zeroes_array.hpp, lines 38-43
void set(size_t index, const T& value)
{
    BB_ASSERT_DEBUG(index >= start_);   // compiled out in release
    BB_ASSERT_DEBUG(index < end_);      // compiled out in release
    data()[index - start_] = value;     // executes unconditionally in release
}

// shared_shifted_virtual_zeroes_array.hpp, lines 86-91
T& operator[](size_t index)
{
    BB_ASSERT_DEBUG(index >= start_);   // compiled out in release
    BB_ASSERT_DEBUG(index < end_);      // compiled out in release
    return data()[index - start_];      // executes unconditionally in release
}
```

`Polynomial::at()` delegates to `operator[]`:

```
// polynomial.hpp, line 268
Fr& at(size_t index) { return coefficients_[index]; }
```

When `index < start_`, the expression `index - start_` wraps around because both are `size_t` (unsigned), producing a huge offset. `data()` returns a pointer to the beginning of the allocated buffer, so `data()[huge_offset]` dereferences an address far beyond the allocation, resulting in a wild pointer write. When `index >= end_` but the backing memory is smaller than `end_ - start_` elements, the access is a classic heap buffer overflow.

Impact: Silent heap corruption in release builds if any caller passes an out-of-range index to a mutable accessor. The `SharedShiftedVirtualZeroesArray` is only accessed through `Polynomial`, which is expected to maintain correct invariants.

This requires a caller bug or a corrupted polynomial range invariant. No current production call site was identified that intentionally writes out of range, but the release-build failure mode is memory corruption rather than a clean abort.

Recommended Mitigation: Promote the mutable-access bounds checks that protect memory safety to always-on `BB_ASSERT`, at least for `set()` and `operator[]`. If profiling shows this is too expensive for hot paths, document the invariant requirement directly on the mutable accessors and add narrow wrapper APIs for checked mutation at call sites where indices are not trivially derived from validated ranges.

Aztec: Acknowledged.

7.2.3 `backing_memory.cpp` preprocessor guard inconsistent with header

Description: In `backing_memory.cpp:20` and `backing_memory.hpp:36`, the preprocessor guards for file-backed storage features differ:

File	Guard
`backing_memory.hpp:36`	`#if !defined(__wasm__) && !defined(_WIN32)`
`backing_memory.cpp:20`	`#if !defined(__wasm__) || defined(ENABLE_WASM_BENCH)`

Two inconsistencies:

1. **Missing `_WIN32` exclusion in `.cpp`** — The header excludes Windows, but the `.cpp` doesn't. On Windows the `.cpp` defines symbols (`storage_budget`, `current_storage_usage`, `parse_size_string`) that nothing can reference because the header never declares them.
2. **`ENABLE_WASM_BENCH` not recognized by header** — The `.cpp` allows WASM builds when `ENABLE_WASM_BENCH` is defined, but the header has no such exception. When building with both `__wasm__` and `ENABLE_WASM_BENCH`, the `.cpp` provides definitions that are invisible to any translation unit including the header.

In UltraHonk production (Linux/macOS), file-backed storage is only activated when `BB_SLOW_LOW_MEMORY=1` is set. Windows and WASM are not supported production targets.

Impact: Informational. No impact on production prover or verifier.

Recommended Mitigation: Make `backing_memory.cpp:20` match `backing_memory.hpp:36`:

```
-#if !defined(__wasm__) || defined(ENABLE_WASM_BENCH)
+#if !defined(__wasm__) && !defined(_WIN32)
```

Aztec: Acknowledged.

7.2.4 `fft_inner_parallel` unsafe when `coeffs == target`

Description: In `polynomial_arithmetic.cpp:86-93`, phase 1 of `fft_inner_parallel` reads from `coeffs[swap_index]` (bit-reversed) and writes to `target[i]` (sequential):

```
Fr::__copy(coeffs[swap_index_1], temp_1);   // read from bit-reversed index
Fr::__copy(coeffs[swap_index_2], temp_2);
target[i + 1] = temp_1 - temp_2;            // write to sequential index
target[i] = temp_1 + temp_2;
```

If `coeffs == target` (same pointer), the bit-reversed read at `swap_index` could access an element that was already overwritten by a previous iteration's sequential write. For example with `n=8`:

```
i=0: writes target[0], target[1]
i=2: reads coeffs[reverse(2)] = coeffs[2] ← OK (not yet written)
     reads coeffs[reverse(3)] = coeffs[6] ← OK
     writes target[2], target[3]
i=4: reads coeffs[reverse(4)] = coeffs[1] ← ALREADY OVERWRITTEN at i=0!
```

The function signature accepts separate `Fr* coeffs` and `Fr* target` pointers but does not assert they are different.

Impact: Informational. In UltraHonk, `ifft` is only called in ZK sumcheck Libra (`zk_sumcheck_data.hpp:233`) and small subgroup IPA (`small_subgroup_ipa.cpp:439`), both on `SUBGROUP_SIZE = 256` with separate input/output buffers. The function signature permits aliasing, and a future caller passing the same buffer would get silently wrong results.

Proof of Concept: `AuditPoC_P32_FftInPlaceAliasing` in `polynomial_arithmetic.test.cpp`:

```
TEST(AuditPoC, P32_FftInPlaceAliasing)
{
    using FF = bb::fr;
    constexpr size_t n = 16;

    auto domain = bb::EvaluationDomain<FF>(n);
    domain.compute_lookup_table();

    std::array<FF, n> coeffs;
    for (size_t i = 0; i < n; i++) {
        coeffs[i] = FF::random_element();
    }

    // Correct: separate input/output buffers
    std::array<FF, n> correct_output;
    std::array<FF, n> coeffs_copy;
    std::copy(coeffs.begin(), coeffs.end(), coeffs_copy.begin());
    polynomial_arithmetic::fft_inner_parallel(
        coeffs_copy.data(), correct_output.data(), domain, domain.root, domain.get_round_roots());

    // Buggy: in-place (coeffs == target)
    std::array<FF, n> aliased;
    std::copy(coeffs.begin(), coeffs.end(), aliased.begin());
    polynomial_arithmetic::fft_inner_parallel(
        aliased.data(), aliased.data(), domain, domain.root, domain.get_round_roots());

    // In-place produces different (wrong) results
    bool mismatch = false;
    for (size_t i = 0; i < n; i++) {
        if (correct_output[i] != aliased[i]) {
            mismatch = true;
            break;
        }
    }
    EXPECT_TRUE(mismatch);
}
```

Recommended Mitigation: In `polynomial_arithmetic.cpp:74`:

```
 void fft_inner_parallel(
     Fr* coeffs, Fr* target, const EvaluationDomain<Fr>& domain, const Fr&, const std::vector<Fr*>&
     ↪  root_table)
 {
+    BB_ASSERT(coeffs != target, "fft_inner_parallel does not support in-place operation");
```

Aztec: Fixed in 80131f6.

Cyfrin: Verified.

7.2.5 `compute_linear_polynomial_product` is O(n^3) instead of O(n^2)

Description: In `polynomial_arithmetic.cpp:213`, `compute_linear_polynomial_product` builds the polynomial `(X - r_0)(X - r_1)···(X - r_{n1})` using a doubly-nested loop with a `compute_sum` call inside, giving total complexity **O(n^3)**:

```
for (size_t i = 0; i < n - 1; ++i) {
    temp = 0;
    for (size_t j = 0; j < n - 1 - i; ++j) {
        scratch_space[j] = roots[j] * compute_sum(&scratch_space[j + 1], n - 1 - i - j);
        //                            compute_sum is O(n - 1 - i - j) → cubic total
        temp += scratch_space[j];
```

```
    }
    dest[n - 2 - i] = temp * constant;
    constant *= Fr::neg_one();
}
```

The same coefficients can be obtained in **O(n^2)** by the standard "multiply by `(X  r_i)` one root at a time" recurrence:

```
dest[0] = -roots[0];
dest[1] = 1;
for (size_t i = 1; i < n; ++i) {
    const Fr r = roots[i];
    dest[i + 1] = dest[i];
    for (size_t k = i; k >= 1; --k) {
        dest[k] = dest[k - 1] - r * dest[k];
    }
    dest[0] = -r * dest[0];
}
```

Short trace with `n = 3`, `roots = [1, 2, 3]`, expected `(X1)(X2)(X3) = X^3  6X^2 + 11X  6` -> `dest = [6, 11, 6, 1]`:

- Init (represents `X  1`): `dest = [1, 1, ?, ?]`.
- Iteration `i = 1` (multiply in `X  2`, `r = 2`):
  - shift `dest[2] = dest[1] = 1`;
  - inner `k=1`: `dest[1] = dest[0]  r·dest[1] = 1  2 = 3`;
  - then `dest[0] = r·dest[0] = 2`.
  - Result: `[2, 3, 1, ?] = (X1)(X2)`.
- Iteration `i = 2` (multiply in `X  3`, `r = 3`):
  - shift `dest[3] = dest[2] = 1`;
  - inner `k=2`: `dest[2] = dest[1]  r·dest[2] = 3  3·1 = 6`;
  - inner `k=1`: `dest[1] = dest[0]  r·dest[1] = 2  3·(3) = 11`;
  - then `dest[0] = r·dest[0] = 3·2 = 6`.
  - Result: `[6, 11, 6, 1] = (X1)(X2)(X3)`.

The inner loop runs high → low so `dest[k1]` still holds its previous-iteration value when read — the shift-then-combine trick that keeps the update in place without a scratch buffer. The current algorithm needs a scratch buffer because `dest[k]` stores the final coefficient `(-1)^(n-k) · e_{n-k}` while its inner loop aggregates partial sums of a different shape; the optimized version keeps the partial polynomial `(X  r_0)···(X  r_i)` directly in `dest[0..i+1]`, so `dest[]` already has the right layout.

As a side benefit, `compute_linear_polynomial_product` is the only caller of `get_scratch_space<Fr>` (polynomial_arithmetic.cpp:21) — the static thread-shared buffer at the root of issue-8. After this fix, `get_scratch_space` can be deleted outright, so one change delivers O(n^3) → O(n^2) cost, zero per-call allocation, and concurrency safety.

Impact: Informational (performance). Production callers via `compute_efficient_interpolation`:

Caller	n
`zk_sumcheck_data.hpp:230` (Libra interpolation, non-BN254 path)	Grumpkin SUBGROUP_SIZE = **87**
`small_subgroup_ipa.cpp:226` (ECCVM challenge polynomial, Grumpkin)	87
`small_subgroup_ipa.cpp:436` (monomial coefficients, Grumpkin)	87

Caller	n
Generic interpolation constructor (any future caller)	up to BN254 SUBGROUP_SIZE = **256**

(BN254 ZK-Sumcheck takes the IFFT path, not this one.)

Proof of Concept: Integrated benchmark `AuditPoC.P36_LinearPolyProductCubicVsQuadratic` in `polynomials/polynomial_arithmetic.test.cpp`. The test generates random roots, runs both implementations, verifies coefficient-by-coefficient equality and that `evaluate(dest, z) == \prod(z  r_i)`, then times 50 runs of each and reports the speedup.

Measured on an M-series Mac, release build:

```
P-36 n=87  (Grumpkin):   current = 307 s    optimized = 35 s    speedup = 8.7×
P-36 n=256 (BN254 subgroup): current = 7.1 ms   optimized = 0.30 ms  speedup = 23.4×
```

The speedup grows linearly with `n` as expected from $O(n^3) \to O(n^2)$.

Removing the scratch buffer also makes `compute_efficient_interpolation` reentrant, so today-sequential call sites become independently parallelizable:

Site
`small_subgroup_ipa.cpp:336-357` — `compute_lagrange_first_and_last` does 2 independent `compute_monomial_coeffici`
Batched proof generation (e.g., Chonk / folding pipelines) — multiple `SmallSubgroupIPAProver::prove()` instances across cir

The mutex inside `get_scratch_space` also gives reviewers a false signal of thread safety, so the fix removes a long-term tripwire.

Recommended Mitigation: Replace the implementation with the incremental "multiply by `(X  r_i)`" loop (shown above). Same output, ~9× faster on production-sized inputs. Then delete `get_scratch_space<Fr>` from `polynomial_arithmetic.cpp` — it has no other callers.

Aztec: Fixed in 044b745.

Cyfrin: Verified.

7.2.6 `UnivariateCoefficientBasis` loses Karatsuba precomputation after arithmetic

Description: In `univariate_coefficient_basis.hpp`, all arithmetic operators (`operator+`, `operator-`, `operator+=`, `operator-=`, scalar operations) return `UnivariateCoefficientBasis<Fr, domain_end, false>` — unconditionally resetting `has_a0_plus_a1` to `false`. This discards the precomputed `a0 + a1` value stored in `coefficients[2]`, even when the result could preserve it cheaply.

```
// operator+ always returns has_a0_plus_a1 = false
template <size_t other_domain_end, bool other_has_a0_plus_a1>
UnivariateCoefficientBasis<Fr, domain_end, false> operator+(
    const UnivariateCoefficientBasis<Fr, other_domain_end, other_has_a0_plus_a1>& other) const
{
    UnivariateCoefficientBasis<Fr, domain_end, false> res(*this);
    res.coefficients[0] += other.coefficients[0];
    res.coefficients[1] += other.coefficients[1];
    // coefficients[2] NOT updated for domain_end=2
    // → precomputed (a0+a1) is lost even though (a0'+a1') = (a0+a1) + (b0+b1)
    return res;
}
```

The in-place operators have the same issue — they skip `coefficients[2]` when `domain_end=2`:

```
// operator+= also returns has_a0_plus_a1 = false
UnivariateCoefficientBasis<Fr, domain_end, false>& operator+=(...)
{
    coefficients[0] += other.coefficients[0];
    coefficients[1] += other.coefficients[1];
    if constexpr (other_domain_end == 3 && domain_end == 3) {
        coefficients[2] += other.coefficients[2];
    }
    // coefficients[2] NOT updated for domain_end=2
    return *this;
}
```

When both operands are `(domain_end=2, has_a0_plus_a1=true)`, the precomputation is preservable for all four operators:

```
operator+/+=:
  this:  a0, a1, (a0+a1)
  other: b0, b1, (b0+b1)
  result: (a0+b0), (a1+b1), (a0+a1)+(b0+b1) = (a0+b0)+(a1+b1) ← correct!

operator-/-=:
  result: (a0-b0), (a1-b1), (a0+a1)-(b0+b1) = (a0-b0)+(a1-b1) ← correct!
```

But the return type is hardcoded to `false`, so downstream `operator*` cannot use the (T,T) Karatsuba branch and must recompute `a0+a1` from scratch.

Impact: Informational (performance). In Sumcheck's inner loop, `UnivariateCoefficientBasis` operations are executed per-gate, per-relation, per-round. The Karatsuba optimization saves 1 field addition per degree-1 multiplication by using the precomputed `a0+a1`.

For relation patterns like `(wire_a + wire_b) * wire_c`:

```
1. wire_a, wire_b, wire_c: converted from Lagrange → (2, true), a0+a1 free
2. wire_a + wire_b → (2, false), Karatsuba precomputation LOST
3. result * wire_c → hits (F,T) branch, recomputes a0+a1 (+1 addition)
```

If step 2 preserved `has_a0_plus_a1=true`, step 3 would use the cheaper (T,T) branch.

Proof of Concept: Measured benchmark:

Integrated benchmark `AuditPoC.P33_AddThenMultiplyKaratsubaPath` in `polynomials/univariate_coefficient_basis.test.cpp`. The test simulates the realistic Sumcheck pattern `(wire_a + wire_b) * wire_c` for 200k iterations:

- Current: `a + b` returns `(2, false)` (precomputation discarded), so `(a+b) * c` takes the (F,T) Karatsuba branch.
- Optimized: manually preserve `a0+a1` in the sum (what the recommended fix would do automatically), so `(a+b) * c` takes the (T,T) branch.

```
TEST(AuditPoC, P33_AddThenMultiplyKaratsubaPath)
{
    constexpr size_t N = 200000;
    std::vector<UnivariateCoefficientBasis<fr, 2, true>> A(N), B(N), C(N);
    // ... fill each with random (a0, a1) and coefficients[2] = a0 + a1

    // Current: (A + B) returns (2, false), so * falls into the (F, T) branch.
    auto t0 = high_resolution_clock::now();
    for (size_t i = 0; i < N; ++i) {
        auto sum = A[i] + B[i];           // has_a0_plus_a1 = false
        auto out = sum * C[i];            // (F, T) Karatsuba branch
    }
    auto t1 = high_resolution_clock::now();
```

```
    // Optimized: reconstruct the sum as (2, true) with coefficients[2] preserved → (T, T).
    for (size_t i = 0; i < N; ++i) {
        UnivariateCoefficientBasis<fr, 2, true> sum;
        sum.coefficients[0] = A[i].coefficients[0] + B[i].coefficients[0];
        sum.coefficients[1] = A[i].coefficients[1] + B[i].coefficients[1];
        sum.coefficients[2] = A[i].coefficients[2] + B[i].coefficients[2];  // a0'+a1'
        auto out = sum * C[i];            // (T, T) Karatsuba branch
    }
    auto t2 = high_resolution_clock::now();
}
```

```
P-33 N=200000:  current ((F,T) chain) = 33.3 ns/op   optimized ((T,T) chain) = 30.2 ns/op   saved = 9.5%
```

Per-`(add + mul)` pattern saved: ~3 ns. The 9.5% is the realistic per-pattern speedup, not the upper bound — it includes the + operation cost as well.

Recommended Mitigation: Template the return type to preserve `has_a0_plus_a1=true` when both inputs are `(2, true)`:

```
template <size_t other_domain_end, bool other_has_a0_plus_a1>
auto operator+(const UnivariateCoefficientBasis<Fr, other_domain_end, other_has_a0_plus_a1>& other)
↪   const
{
    constexpr bool preserve = (domain_end == 2 && other_domain_end == 2
                               && has_a0_plus_a1 && other_has_a0_plus_a1);
    UnivariateCoefficientBasis<Fr, domain_end, preserve> res;
    res.coefficients[0] = coefficients[0] + other.coefficients[0];
    res.coefficients[1] = coefficients[1] + other.coefficients[1];
    if constexpr (other_domain_end == 3 && domain_end == 3) {
        res.coefficients[2] = coefficients[2] + other.coefficients[2];
    } else if constexpr (preserve) {
        res.coefficients[2] = coefficients[2] + other.coefficients[2];
    }
    return res;
}
```

Similar changes needed for `operator-`, `operator+=`, `operator-=`. Profiling recommended before applying.

Aztec: Acknowledged.

7.2.7 `Polynomial::full()` performs unnecessary double-clone

Description: `Polynomial::full()` clones the polynomial's backing memory **twice**. The first clone is immediately discarded by the second:

```
// polynomial.cpp:298
template <typename Fr> Polynomial<Fr> Polynomial<Fr>::full() const
{
    Polynomial result = *this;                                          // ← Clone 1 (wasted)
    result.coefficients_ = _clone(coefficients_, virtual_size() - end_index(), start_index());
                                                                        // ← Clone 2 (the kept one)
    return result;
}
```

`Polynomial` has only one data member: `coefficients_`. So:

1. Line 244 (`Polynomial result = *this;`) invokes the copy constructor, which delegates to `Polynomial(const Polynomial& other, size_t target_size)`. That constructor calls `_clone(other.coefficients_, 0)` — a full deep copy of all `other.size()` backed elements (allocation + memcpy).

2. Line 246 (`result.coefficients_ = _clone(...)`) calls `_clone` again on the original `this->coefficients_` and assigns the result. This drops the `shared_ptr` to the line 244 allocation (refcount → 0 → `delete[]`) and installs a new allocation.

The line 244 clone's data is never used. Both the allocation and the memcpy that filled it are wasted.

Impact: Informational, but the IPA-prover impact is non-trivial.

`full()` is called only once per IPA prove (`ipa.hpp:182`) — but on a polynomial sized to the **circuit size**:

Circuit size `N`	Wasted alloc + memcpy per IPA prove
2^{16} (test bench)	~2 MB
2^{20} (typical app circuit)	**~32 MB**
2^{28} (`CONST_SIZE_PROOF_LOG_N` upper bound)	**~8 GB**

For typical UltraHonk circuits, this is **~32 MB of extra alloc + memcpy + free on every IPA prove** — real memory-bandwidth and allocator pressure on a hot prover path. The fix is a one-line change with zero correctness risk.

The author of `full()` routed through `Polynomial result = *this` (itself going through `_clone` via the copy constructor) and then called `_clone` again directly. Each step looks correct in isolation; the duplication is only visible when both calls are traced together.

Proof of Concept: Integrated test `AuditPoC_P48_FullDoubleClone` in `polynomials/polynomial.test.cpp`. It compares the time taken by `full()` against a baseline single-clone operation (the copy constructor) on the same-size polynomial:

```
TEST(Polynomial, AuditPoC_P48_FullDoubleClone)
{
    using FF = bb::fr;
    using Polynomial = bb::Polynomial<FF>;

    const size_t SIZE = 1 << 16;  // 65536 elements (~2 MB of Fr data)
    auto poly = Polynomial::random(SIZE, SIZE, 0);

    const int ITERATIONS = 100;

    // Time full() - expected to do TWO clones
    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < ITERATIONS; ++i) {
        auto result = poly.full();
        (void)result;
    }
    auto t1 = std::chrono::steady_clock::now();
    auto full_us = std::chrono::duration_cast<std::chrono::microseconds>(t1 - t0).count();

    // Time single-copy via copy constructor - does ONE clone
    auto t2 = std::chrono::steady_clock::now();
    for (int i = 0; i < ITERATIONS; ++i) {
        Polynomial result = poly;
        (void)result;
    }
    auto t3 = std::chrono::steady_clock::now();
    auto single_us = std::chrono::duration_cast<std::chrono::microseconds>(t3 - t2).count();

    const double ratio = static_cast<double>(full_us) / static_cast<double>(single_us);
    EXPECT_GT(ratio, 1.3) << "full() should be noticeably slower due to the extra _clone call";
}
```

Observed output on a release build (M-series Mac):

```
P-48: full()         took 7263 us over 100 iterations
P-48: single-copy    took 3001 us over 100 iterations
P-48: ratio = 2.42x (expected ~2x if double-copy is real)
P-48 CONFIRMED: full() does more work than a single copy - consistent with double-clone
```

A ratio of ~2x is empirical proof that `full()` performs two clones. The slight overshoot above 2.0 is consistent with the extra `new`/`delete` pair from the discarded allocation (allocator overhead beyond the pure memcpy cost).

```
cd barretenberg/cpp/build && ninja polynomials_tests
./bin/polynomials_tests --gtest_filter="*AuditPoC_P48*"
```

Recommended Mitigation: Skip the first clone by constructing `result` empty, then assigning the one clone we actually need:

```
template <typename Fr> Polynomial<Fr> Polynomial<Fr>::full() const
{
    Polynomial result;   // default-constructed, empty (no allocation)
    result.coefficients_ = _clone(coefficients_, virtual_size() - end_index(), start_index());
    return result;
}
```

`Polynomial` has a default constructor (`= default`) that leaves `coefficients_` in a harmless empty `SharedShiftedVirtualZeroesArray` state (default-initialized members, no allocations). The assignment then installs the only clone we actually want.

This eliminates one `new Fr[size]` + one `memcpy(size * sizeof(Fr))` + one `delete[]` per `full()` call. For size-2^{20} polynomials, that's ~32 MB of saved work.

Aztec: Fixed in 27400b6.

Cyfrin: Verified.

7.2.8 scalar `operator*=` unnecessarily restricted by `requires(!has_a0_plus_a1)`

Description: In `univariate_coefficient_basis.hpp:232`, `operator*=(const Fr& scalar)` has a `requires(!has_a0_plus_a1)` constraint that prevents it from being called on `(domain_end=2, has_a0_plus_a1=true)` objects:

```
UnivariateCoefficientBasis<Fr, domain_end, false>& operator*=(const Fr& scalar)
    requires(!has_a0_plus_a1)   // ← unnecessarily restrictive
{
    coefficients[0] *= scalar;
    coefficients[1] *= scalar;
    if constexpr (domain_end == 3) {
        coefficients[2] *= scalar;
    }
    return *this;
}
```

Unlike `operator+=` and `operator-=` with scalars — where the constraint IS necessary because adding/subtracting a scalar changes `a` without changing `a`, invalidating the precomputed `a + a` — scalar multiplication scales all coefficients uniformly:

```
Before: a, a, (a + a)
After:  s·a, s·a, s·(a + a) = s·a + s·a  ← still valid!
```

Comparison of scalar operators:

Operator	`requires(!has_a0_plus_a1)` justified?	Reason
`operator+=(scalar)`	**Yes**	`a` changes, `a` doesn't → `a+a` invalidated
`operator-=(scalar)`	**Yes**	Same as above
`operator*=(scalar)`	**No**	All terms scale by `s` → `s·(a+a)` still valid

The same issue applies to the non-in-place `operator*(const Fr& scalar)` which returns `has_a0_plus_a1=false` and does not scale `coefficients[2]`.

Impact: Informational. This is a conservative design choice that simplifies the type system. The performance impact is the same as issue-22: downstream `operator*` falls into a more expensive Karatsuba branch when the precomputation has been discarded.

Proof of Concept: Measured benchmark:

Integrated benchmark `AuditPoC.P34_KaratsubaTTvsFFBranch` in `polynomials/univariate_coefficient_basis.test.cpp`. The test directly compares the cost of the (T,T) Karatsuba branch (which P-34 unlocks downstream) against the (F,F) branch (which the current code is forced into after a scalar `*=` strips the precomputation):

```
TEST(AuditPoC, P34_KaratsubaTTvsFFBranch)
{
    constexpr size_t N = 200000;
    std::vector<UnivariateCoefficientBasis<fr, 2, true>>  lhs_T(N), rhs_T(N);
    std::vector<UnivariateCoefficientBasis<fr, 2, false>> lhs_F(N), rhs_F(N);
    // ... fill both representations with the same random (a0, a1) / (b0, b1)
    //     - the (2, true) copies additionally set coefficients[2] = a0 + a1.

    // (T, T) - the cheap branch P-34 would unlock by removing the `requires` gate.
    auto t0 = high_resolution_clock::now();
    for (size_t i = 0; i < N; ++i) {
        auto r = lhs_T[i] * rhs_T[i];     // (T, T) Karatsuba branch
    }
    auto t1 = high_resolution_clock::now();

    // (F, F) - what the current code is forced into once a scalar `*=` strips the flag.
    for (size_t i = 0; i < N; ++i) {
        auto r = lhs_F[i] * rhs_F[i];     // (F, F) Karatsuba branch
    }
    auto t2 = high_resolution_clock::now();
}
```

```
P-34 N=200000:  (T,T) Karatsuba = 25.6 ns/op   (F,F) Karatsuba = 29.7 ns/op   (F,F) overhead = 13.7%
```

Per-multiplication cost difference: ~4 ns (one extra field addition). The 13.7% overhead is the upper bound on what the P-34 fix would save for any multiplication chain that currently goes through (F,F) because of an intervening scalar `*=`.

Recommended Mitigation: Remove the `requires` constraint and preserve `has_a0_plus_a1`:

```
- UnivariateCoefficientBasis<Fr, domain_end, false>& operator*=(const Fr& scalar)
-     requires(!has_a0_plus_a1)
+ UnivariateCoefficientBasis& operator*=(const Fr& scalar)
  {
      coefficients[0] *= scalar;
      coefficients[1] *= scalar;
      if constexpr (domain_end == 3) {
          coefficients[2] *= scalar;
+     } else if constexpr (has_a0_plus_a1) {
+         coefficients[2] *= scalar;  // preserve a+a precomputation
```

```
    }
    return *this;
}
```

Aztec: Acknowledged.